Trust-Aware Task Allocation in Dynamic Rover Networks with Constrained QUBO Edge Pruning: a Complete Project Report

[**TO FILL: author names, one per line**]

[**TO FILL: submission date**]

Abstract

[**TO FILL: abstract text; 150 to 250 words summarizing motivation, pipeline, verification approach, headline results, and the honest limitations**]

Contents

1 Introduction **4**
- 1.1 Motivation . . . 4
- 1.2 Problem statement . . . 4
- 1.3 Why a QUBO, and why we say so honestly . . . 5
- 1.4 The mist, fog, and cloud deployment vision . . . 5
- 1.5 An audit-first philosophy . . . 5
- 1.6 Contributions . . . 6
- 1.7 The per-cycle pipeline . . . 6
- 1.8 Report roadmap . . . 6

2 Background and Foundations **7**
- 2.1 Dynamic Graphs and Rover Networks . . . 7
- 2.2 Constrained Combinatorial Optimization and the QUBO Form . . . 8
  - 2.2.1 Penalty encoding of hard constraints . . . 9
- 2.3 Simulated and Quantum Annealing . . . 9
  - 2.3.1 Simulated annealing . . . 9
  - 2.3.2 Quantum annealing and simulated quantum annealing . . . 10
- 2.4 Mixed-Integer Linear Programming and Exact Ground Truth . . . 10
- 2.5 Message-Passing Graph Neural Networks . . . 11
- 2.6 Trust Models for Multi-Agent Systems . . . 11
- 2.7 Statistical Tools Used Throughout the Report . . . 12
- 2.8 Notation . . . 12

3 System Architecture **13**
- 3.1 The Three-Tier Deployment Model . . . 13
- 3.2 Per-Cycle Data Flow . . . 13
- 3.3 Component Inventory . . . 15
- 3.4 Design Rules . . . 16
- 3.5 Summary . . . 16

4 The Constrained QUBO Formulation **16**
4.1 Candidate edges and features . . . 17
4.2 Decision variables . . . 17
4.3 Soft objective . . . 18
4.4 Hard constraints as dominant penalties . . . 18
4.5 Input validation at construction . . . 20
4.6 A worked toy example . . . 20
4.7 Verification of the energy function . . . 20
4.8 Summary . . . 21

5 Solvers: from Bit Flips to Feasibility Projection **21**
5.1 A bit-level simulated annealer . . . 21
5.2 Why bit-level annealing fails on constrained QUBOs . . . 22
5.3 Sampler benchmark on the real Q matrices . . . 22
5.4 The production solver: feasibility-projected edge annealing . . . 23
5.5 Defaults, measured cost, and accuracy . . . 24
5.6 An honest framing . . . 25

6 Trust Modeling and the Trust-Aware GNN **25**
6.1 Deployment constraint: mist, fog, and cloud tiers . . . 25
6.2 Node features . . . 25
6.3 The trust model . . . 26
6.4 One round of message passing on the pruned graph . . . 26
6.5 Assignment rule . . . 27
6.6 Cloud readout training: a null result . . . 27
6.7 Ablation: what actually carries the performance . . . 28

7 The Causal Simulation Harness **28**
7.1 Per-Cycle Phases . . . 29
7.2 Hidden Ground Truth . . . 30
7.3 The Causal Delivery Model . . . 30
7.4 Default Configuration . . . 31
7.5 Fault Injection and Edge Lifecycle . . . 31
7.6 Cloud Amortization and Staleness . . . 32
7.7 Metrics . . . 32
7.8 Statistical Protocol . . . 32
7.9 Run Modes . . . 32

8 Exact Baselines: MILP, Linearized QUBO, and Enumeration **33**
8.1 Why exact ground truth is necessary . . . 33
8.2 Direct MILP solution of the original problem . . . 34
8.3 Exact linearized solution of the QUBO matrix . . . 34
8.4 Exhaustive enumeration at small scale . . . 35
8.5 Cross-verification of the three mechanisms . . . 35
8.6 Engineering context . . . 35
8.7 The headline comparison . . . 36

9 Scaling Analysis **37**
9.1 Experimental design . . . 37
9.2 How the variables grow . . . 37
9.3 Scope limits and how we report them . . . 38
9.4 Solution quality at the largest sizes . . . 39
9.5 Inference latency is not the bottleneck . . . 39

9.6 Implications for the tier split . . . 39

10 Testing Methodology: the Hostile Audit Suite **40**
10.1 Philosophy: an auditor, not a cheerleader . . . 40
10.2 Ground-truth discipline . . . 40
10.3 The fourteen sections . . . 41
10.4 Engine and server tests for the mission console . . . 44
10.5 What this methodology buys . . . 44

11 Results and Discussion **44**
11.1 Solver audit on the encoded Q matrix . . . 45
11.2 Production solver quality . . . 45
11.3 The MILP counterpoint . . . 46
11.4 Mission ablations . . . 46
11.5 The pruner negative result . . . 47
11.6 Staleness and trust tracking . . . 47
11.7 The training null result . . . 48
11.8 Discussion: what the pipeline is actually good for . . . 48

12 Physics Simulation: the Gazebo Integration **49**
12.1 Software Stack and Deployment . . . 49
12.2 Interfaces: Bridging Gazebo and the Planner . . . 49
12.3 The Planner Node Cycle . . . 50
12.4 World and Rover Model . . . 51
12.5 Limitations and Planned Remedies . . . 52
12.6 Running the Demonstration . . . 52

13 The Custom three.js Simulation Engine **53**
13.1 Overview and Design Principles . . . 53
13.2 Two Processes, One Command . . . 53
13.3 The Shared Configuration . . . 54
13.4 Honest Comparison by Construction . . . 55
13.5 The Engine Core . . . 55
13.6 The Render Layer . . . 56
13.7 State, Planner Loop, and User Interface . . . 57
13.8 Planned Engine Extensions . . . 57

14 Conclusion and Future Work **57**
14.1 What was built . . . 57
14.2 What was proven . . . 58
14.3 What was disproven or humbled . . . 58
14.4 Lessons . . . 59
14.5 Future work . . . 59
14.6 Closing remarks . . . 59
14.7 Pending items in this report . . . 59

References **60**

A Pending Fields and Placeholders **60**

1 Introduction

1.1 Motivation

Teams of small, inexpensive rovers are an increasingly realistic answer to a class of problems where a single large machine is the wrong tool. In planetary exploration, a cluster of metre-scale scouts can survey a lava tube or a crater rim in parallel, covering in hours what one flagship rover covers in weeks, and the loss of one unit degrades the mission instead of ending it. Closer to home, disaster response presents the same structure: after an earthquake or a chemical release, a team of disposable ground robots can map voids, sample air, and relay survivor signals through rubble that no human and no single large vehicle can enter. Both settings share three properties that drive everything in this report. The environment is unknown and changing, the robots are cheap and therefore individually unreliable, and there is no infrastructure: the team must carry its own network with it.

That network is the central object of our study. We consider rovers that communicate over short-range, low-power radios of the ESP-NOW class: link budgets of a few hundred milliwatts, useful ranges of tens of metres, and packet delivery probabilities that fluctuate with distance, occlusion, and interference. As the rovers move, the set of pairs that can hear each other changes continuously, so the communication topology is not a fixed design choice but a *dynamic graph* that the mission software inherits anew every control cycle. Two consequences follow. First, the full graph is rarely the graph you want to compute on: it contains weak, stale, and expensive links that add noise to every downstream decision while consuming radio and processor budget. Second, the rovers themselves differ in dependability, and that dependability drifts. A unit with a degrading motor controller or a corrupt sensor may still look perfectly reachable on the radio graph while quietly failing every task it is given.

These observations motivate the two mechanisms at the heart of the project. *Link pruning* treats the working communication graph as a decision variable: rather than accepting whatever topology the radios happen to provide, the system selects, once per cycle, a smaller subgraph that is provably connected where it matters, respects per-rover degree and processing budgets, and favours links that are trusted, fresh, and cheap. *Trust* treats reliability as a learned quantity: each rover carries a trust score, updated online from observed task outcomes, which biases both which links survive pruning and which rover is offered which task. A rover that keeps failing is not argued with; it is simply routed around, until its later successes earn its way back. The two mechanisms are coupled, and the coupling is the point: pruning decides who talks to whom, trust decides who is believed, and both feed a task allocator whose successes and failures become the evidence for the next round of trust updates.

1.2 Problem statement

Each control cycle, the system faces two coupled discrete decisions. The first is *which links to keep*. Given the current dynamic graph, a vector of per-link features (trust of the endpoints, measured link quality, battery levels, proximity, freshness of the last successful exchange, criticality of pending traffic, and the processing cost of maintaining the link), and a set of hard requirements (mission-critical nodes must remain reachable from a designated root, no rover may exceed its degree limit, and the total processing cost may not exceed a global budget), choose a subset of edges that maximises a weighted utility while satisfying every hard requirement. The second decision is *which rover does which task*. Given the pruned graph, score every rover for every pending task and produce an assignment that respects per-rover capacity. The decisions are coupled because the scoring runs *on the pruned graph*: a rover cut off by pruning cannot be reached by the assignment, and an assignment that concentrates load changes the trust and battery features that the next pruning round will see. A deliberate design rule follows from this coupling: connectivity is never sacrificed for utility, since a beautifully scored task allocation is worthless if the winning rover cannot be sent its orders.

The problem is a constrained combinatorial optimisation with quadratic structure (the utility of keeping two links is not always the sum of their individual utilities, because redundant paths and shared endpoints interact), and it must be solved repeatedly, online, under a time budget, on hardware far weaker than a workstation.

1.3 Why a QUBO, and why we say so honestly

We encode the link-selection decision as a *quadratic unconstrained binary optimisation* problem, or QUBO: an energy function over binary variables in which every hard constraint becomes a penalty term large enough that violating it is never worth the utility it could buy. The attraction of this encoding is portability. A QUBO is the native input format of quantum annealers and of a growing family of dedicated classical samplers, so a single formulation can be submitted unchanged to a D-Wave-class device, to the D-Wave `neal` simulated annealer, to a tabu search, or to an Ising-machine simulator such as OpenJij. If annealing hardware matures, the encoding is already waiting for it.

Honesty, however, is a design requirement of this project, not a garnish. On classical hardware, an exact mixed-integer linear program solved with CBC finds provably optimal solutions to the same constrained problem roughly two orders of magnitude faster than our annealer, and the off-the-shelf QUBO samplers we benchmarked find the exact optimum on our actual penalty matrix in only about half to two thirds of instances. We report these numbers in full, in the body of the report, with the same prominence as our positive results. The QUBO formulation is justified by its hardware trajectory and its clean separation of modelling from solving, not by a false claim of classical superiority. Where a classical exact method wins, we say it wins.

1.4 The mist, fog, and cloud deployment vision

The architecture is organised around three computational tiers. The *mist* tier is the rovers themselves: microcontrollers with just enough compute to run radio exchange, local sensing, and task execution. The *fog* tier is a single Raspberry Pi-class board riding with the team. It runs the two latency-critical inference stages each cycle: the production annealer that solves the pruning QUBO, and a deliberately tiny trust-aware message-passing graph neural network (GNN), implemented in pure NumPy, that scores rovers on the pruned graph. Nothing on the fog tier requires a library heavier than NumPy, because nothing on that tier may assume a network connection. The *cloud* tier, reachable opportunistically, does the heavy and patient work: training the GNN readout weights on collected mission logs, and amortising expensive solves so that a fresh QUBO need not be built from scratch every cycle. This division is not decorative. It dictates engineering constraints that the reader will see resurface throughout the report: the GNN must run without a deep-learning framework, the annealer must be restartable and seedable for reproducibility, and every cloud dependency must have a graceful fog-tier fallback.

1.5 An audit-first philosophy

This project is built as an audit of itself. Every quantitative claim in this report is backed by an executable check, and the check survives in the project's hostile test harness long after the claim is written. The QUBO encoding is verified against exhaustive enumeration of all feasible edge subsets on small instances and against an exact MILP on larger ones, so the penalty matrix is never taken on faith. Four off-the-shelf QUBO samplers are benchmarked directly on the penalty matrix our formulation actually produces, rather than on a convenient synthetic instance. Statistical claims about mission success come with paired bootstrap confidence intervals and permutation p-values, and comparisons that fail to reach significance are labelled as such in the tables where they appear. Most importantly, negative results are reported with equal prominence. Our learned GNN readout does not beat its hand-tuned weights; the QUBO pruner loses on raw mission success to a trivially simple threshold pruner at one fleet scale; classical MILP dominates

the annealer. Each of these findings is presented, explained, and kept, because a report about trustworthy robots that hides its own failures would be making the opposite of its point.

1.6 Contributions

The project makes the following contributions.

- A constrained QUBO formulation of communication-graph pruning for dynamic rover networks, unifying multi-feature link utility with hard connectivity, degree, and processing-budget constraints in a single annealer-compatible energy function.
- A feasibility-projected simulated annealer that moves only in the space of valid edge sets, repairing every candidate against the hard constraints, and that matches exact enumeration on small instances and achieves 95% exactness against MILP ground truth on larger ones.
- A solver-level audit benchmarking four off-the-shelf QUBO samplers (bit-level simulated annealing, D-Wave `neal`, tabu search, and an OpenJij-based Ising sampler) on the actual penalty matrix, quantifying how far generic samplers fall short of feasibility-aware search.
- A trust-aware GNN scoring stage, small enough for pure NumPy inference on Raspberry Pi-class fog hardware, integrated with online per-rover trust updates driven by observed task outcomes.
- A causal end-to-end simulation harness in which hidden ground-truth reliability drives outcomes, so that mission success genuinely depends on the pruned topology rather than merely correlating with it, with paired bootstrap and permutation statistics throughout.
- A physics integration using Gazebo Harmonic and ROS 2 Jazzy, plus a browser-based fleet console built on a custom three.js engine with a Python backend, allowing live inspection and in-place tuning of the full pipeline.
- A set of deliberately reported negative results, including the null outcome of readout training and the raw-success deficit of QUBO pruning against a simple threshold baseline, each analysed rather than suppressed.

1.7 The per-cycle pipeline

Figure 1 summarises the loop that every chapter of this report elaborates. Each control cycle begins from the current dynamic communication graph. The QUBO builder encodes the pruning decision as a penalty matrix, and the production annealer solves it in feasibility-projected edge space, yielding the working graph. The trust-aware GNN scores rovers on that working graph, a capacity-aware greedy stage assigns the pending tasks, and outcomes are observed. The trust model absorbs the outcomes as evidence, updates every rover's score, and the updated features feed the next cycle's graph construction, closing the loop.

1.8 Report roadmap

The report is organised into fourteen chapters, summarised in Table 1 and sketched here. Chapter 1 is the present chapter. Chapter 2 reviews related work on multi-robot task allocation, trust models for robotic teams, and QUBO and annealing methods, positioning our combination of the three. Chapter 3 fixes notation and formalises the system model: the dynamic graph, the link and node features, and the two coupled per-cycle decisions. Chapter 4 derives the constrained QUBO formulation term by term, including the penalty-domination argument that makes hard constraints safe. Chapter 5 presents the production annealer, its feasibility projection, spanning-tree restarts, and polish phase, and analyses its evaluation budget. Chapter 6 reports the solver

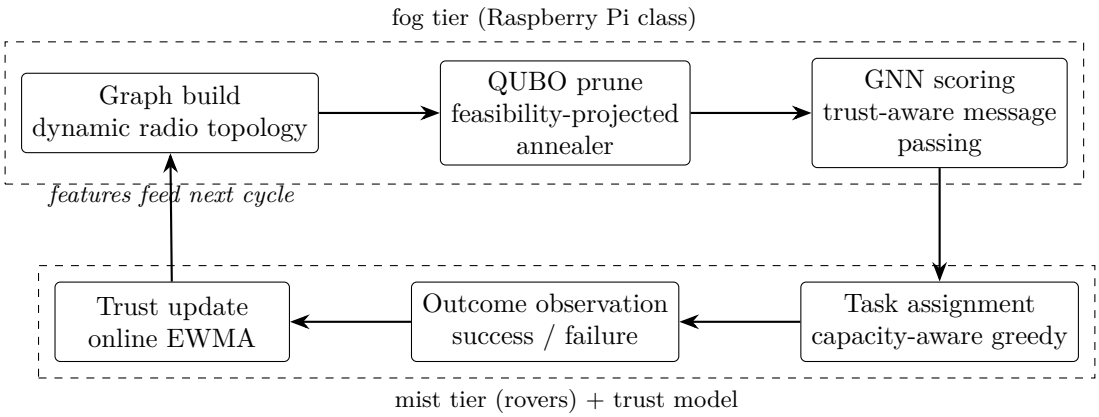


Figure 1: The per-cycle pipeline. Each control cycle prunes the dynamic communication graph with a constrained QUBO, scores rovers with a trust-aware GNN on the pruned graph, assigns tasks under capacity limits, observes outcomes, and folds the evidence back into the trust model that shapes the next cycle's features.

audit: exact enumeration and MILP ground truth, the four off-the-shelf samplers, and the sensitivity sweeps. Chapter 7 describes the trust model and the trust-aware GNN, including the null result on learned readout weights. Chapter 8 specifies the causal simulation harness, the hidden reliability field, and the statistical machinery. Chapter 9 gives the mission-level results: ablations, pruner comparisons, staleness margins, and the negative results. Chapter 10 covers the mist, fog, and cloud deployment architecture and its engineering constraints. Chapter 11 presents the Gazebo Harmonic and ROS 2 Jazzy physics integration. Chapter 12 describes the browser-based fleet console, its three.js rendering engine, and its live tuning interface. Chapter 13 collects the threats to validity and the limitations we consider binding. Chapter 14 concludes and maps the most promising directions for future work.

2 Background and Foundations

This chapter collects the mathematical and algorithmic machinery on which the rest of the report rests. Our aim is not to survey any single field exhaustively, but to build a shared vocabulary: what a dynamic rover network is, why its task-allocation problem is combinatorial, how the QUBO form encodes that problem, which solvers we use to attack it, and how we judge the answers with exact methods and statistics. Each later chapter cites back to the definitions given here.

2.1 Dynamic Graphs and Rover Networks

We model the fleet at any instant as an undirected graph $G_t = (V, E_t)$, where each vertex $v \in V$ is a rover and an edge $\{u, v\} \in E_t$ exists whenever rovers u and v are within radio range of one another. Because the rovers move, the edge set E_t changes from cycle to cycle: rovers that could exchange messages one minute may be out of range the next, and links appear or vanish as the fleet spreads out, regroups, or loses a rover to a fault. A graph whose edge set is a function of time in this sense is a *dynamic graph*.

Dynamism has three practical consequences for our project. First, decisions are perishable: a routing or allocation plan computed on G_t may be invalid on G_{t+1} even if no rover has failed, simply because the topology moved on. Second, every decision must be recomputed on a deadline; a solver that is exact but too slow is useless if the graph has changed by the time it finishes. Third, information is local and stale: a rover knows its own state precisely but learns about distant rovers only through relayed messages, so any score or estimate carried through the network has

Table 1: Structure of the report.

Chapter	Title	Topic
1	Introduction	Motivation, problem statement, contributions, roadmap
2	Related work	Task allocation, trust models, QUBO and annealing
3	System model	Dynamic graph, features, coupled decisions, notation
4	QUBO formulation	Utility terms, hard constraints, penalty domination
5	Production annealer	Feasibility projection, restarts, polish, budget
6	Solver audit	Enumeration, MILP, off-the-shelf samplers, sensitivity
7	Trust and the GNN	Trust updates, message passing, readout null result
8	Simulation harness	Causal outcomes, hidden reliability, statistics
9	Mission results	Ablations, pruner comparison, negative results
10	Deployment tiers	Mist, fog, and cloud architecture and constraints
11	Physics integration	Gazebo Harmonic and ROS 2 Jazzy testbed
12	Fleet console	three.js engine, Python backend, live tuning
13	Threats to validity	Limitations, assumptions, scope of claims
14	Conclusion	Summary of findings and future work

a freshness that decays with time. The pipeline in this project is built around these three facts. Each cycle takes the current graph G_t, prunes it to a sparser subgraph that still supports the mission, scores the rovers on the pruned graph, assigns tasks, and folds the observed outcomes back into a trust model that shapes the next cycle's decisions.

Two features of the graph matter throughout. The first is *connectivity*: a pruned subgraph is useful only if every rover that must receive a task remains reachable from a designated root rover, which we formalize below as a flow requirement. The second is *degree*: a rover with a low-power radio cannot maintain arbitrarily many simultaneous links, so each vertex carries a degree limit. Both features are hard constraints, not preferences, and they drive the constrained optimization formulation of the next section.

2.2 Constrained Combinatorial Optimization and the QUBO Form

Many planning problems ask us to choose a subset of objects (edges, routes, assignments) that maximizes some utility while respecting limits (budget, capacity, connectivity). Because the choices are discrete, these are *combinatorial optimization* problems, and most interesting instances are NP-hard: no algorithm is known that solves every instance in time polynomial in the input size.

A quadratic unconstrained binary optimization problem, or *QUBO*, is the following canonical form. Let $z \in \{0,1\}^n$ be a vector of binary decision variables, $Q \in \mathbb{R}^{n\times n}$ a symmetric matrix, and $c \in \mathbb{R}$ a constant offset. The objective is

$$E(z) \;=\; z^\top Q z + c \;=\; \sum_i Q_{ii}\, z_i \;+\; \sum_{i<j} Q_{ij}\, z_i z_j \;+\; c, \tag{1}$$

and we seek $z^\star = \arg\min_z E(z)$. The diagonal entries Q_{ii} price each variable on its own; the off-diagonal entries Q_{ij} price pairs, rewarding or penalizing the co-selection of i and j. Since

$z_i^2 = z_i$ for binary variables, linear terms fold into the diagonal. The QUBO form is attractive for two reasons: it is the native input format of quantum annealers and of most generic binary samplers, and it forces the modeler to make every trade-off explicit as a number in one matrix.

2.2.1 Penalty encoding of hard constraints

A QUBO is *unconstrained*, so the hard constraints of the underlying problem must be folded into the objective. The standard device is a *penalty term*: for each constraint, we add to Q a quadratic form that is zero exactly when the constraint holds and is strongly positive otherwise. For example, a degree limit $\sum_{j\sim i} z_{ij} \le d_i$ at vertex i becomes a penalty $P\big(\max\{0, \sum_{j\sim i} z_{ij} - d_i\}\big)^2$ after introducing integer slack variables, themselves encoded as a short binary expansion. In this project we encode four constraint families this way: per-rover degree limits, a global edge budget, single-commodity flow conservation for connectivity, and flow capacity on each edge.

The penalty weight P must be chosen so that *breaking a constraint is never worth it*. Formally, let Δ_{soft} be the largest possible swing of the soft (constraint-free) part of the objective between any two candidate solutions, and let $g_{\min} > 0$ be the smallest nonzero violation of any constraint. Then any $P > \Delta_{\text{soft}}/g_{\min}$ guarantees that every minimizer of E is feasible. If P is too small, the sampler will happily buy soft utility at the price of an infeasible graph; if P is needlessly huge, the penalty terms flatten the energy landscape into plateaus separated by cliffs, which slows heuristic search. In this project the soft terms are normalized so that one selected edge is worth at most about one unit of utility, while a single constraint violation costs at least 100 units. The gap of two orders of magnitude places every feasible solution strictly below every infeasible one, and our invariant tests verify this dominance property directly on the assembled matrix Q.

2.3 Simulated and Quantum Annealing

2.3.1 Simulated annealing

Simulated annealing (SA) is a local-search heuristic inspired by the slow cooling of a solid. Starting from an initial candidate z, it repeats the following move: propose a neighbor z' (for instance by flipping one bit), compute the energy change $\Delta E = E(z') - E(z)$, and accept the move with the *Metropolis criterion*

$$\Pr(\text{accept}) \;=\; \begin{cases} 1, & \Delta E \le 0, \\ \exp(-\Delta E / T), & \Delta E > 0, \end{cases} \tag{2}$$

where T is a control parameter called the *temperature*. Downhill moves are always taken; uphill moves are taken with a probability that shrinks both with the size of the climb and as T falls. High temperature early in the run lets the search wander across valleys and escape shallow local minima; low temperature late in the run freezes it into a local minimum, ideally a deep one. The schedule $T_k = T_0\, r^k$ with cooling rate r slightly below one is the workhorse choice and the one used throughout this project; Figure 2 shows the shape. Because a single run is a gamble on its starting point, practical implementations use *restarts*: several independent runs from different initial states, keeping the best result. All annealers in this report also expose a random seed, which lets us reproduce every experiment exactly.

Plain bit-flip SA on a penalty-laden QUBO has a structural weakness: most bit strings are infeasible, and penalty cliffs separate feasible regions, so a random walk spends its budget among infeasible states. This observation motivates the *projected* annealer developed later in this report, whose moves live entirely in the space of feasible edge sets. Chapter 4 describes it; here we note only that the Metropolis rule is unchanged and the projection enters in how neighbors are proposed.

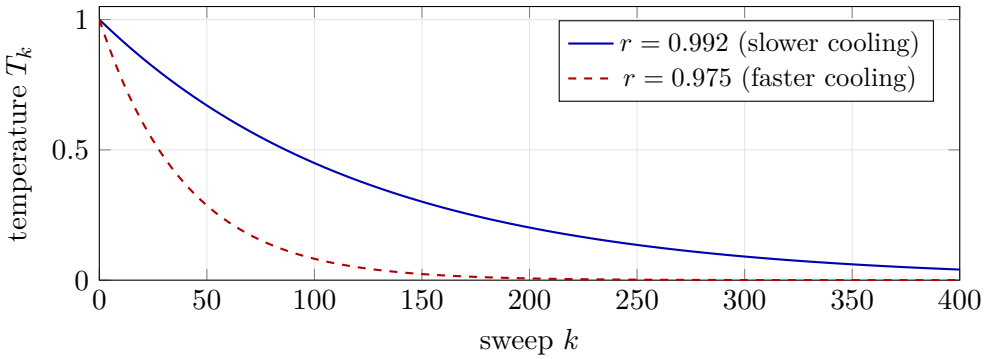


Figure 2: Illustrative exponential cooling schedules $T_k = T_0\, r^k$ with $T_0 = 1$ for two cooling rates. Slower cooling explores longer before freezing; faster cooling reaches low temperatures sooner but risks quenching into a poor local minimum. The curves are schematic and do not correspond to a specific solver run.

2.3.2 Quantum annealing and simulated quantum annealing

Quantum annealing replaces thermal hopping over energy barriers with quantum tunneling through them. Physically, a system of coupled qubits is prepared in the ground state of a simple *driver* Hamiltonian and then slowly morphed into a *problem* Hamiltonian whose ground state encodes the QUBO optimum; if the evolution is slow enough and hardware noise permits, the system ends in or near the optimum. We do not have quantum hardware in this project. Instead we use *simulated quantum annealing* (SQA), a classical path-integral Monte Carlo method that mimics the quantum system: the bit string is replicated into several coupled *Trotter slices*, and a transverse field parameter plays the role of temperature, being annealed downward over the run. SQA runs on ordinary hardware and, like thermal SA, offers no optimality certificate, so it enters our study as one more sampler to audit, alongside a local bit-flip SA we implement ourselves, a production-grade SA library, and a tabu search library.

2.4 Mixed-Integer Linear Programming and Exact Ground Truth

Heuristic samplers tell us the best energy they found; they never tell us how far that energy is from the true optimum. To measure that gap we need an exact solver, and ours comes from *mixed-integer linear programming* (MILP). A MILP minimizes a linear objective subject to linear constraints, with some variables required to be integer. The workhorse algorithm is branch and bound with linear-programming relaxations, and we use the open source solver CBC through a Python modeling layer.

Two exact models serve as ground truth in this project. The first is a MILP of the original edge-selection problem itself, with binary edge variables, integer slacks, and continuous flow variables. The second is an exact solve of the assembled QUBO matrix via the standard linearization: each quadratic term $z_i z_j$ with $Q_{ij} \neq 0$ is replaced by an auxiliary binary variable y_{ij} plus the constraints $y_{ij} \leq z_i$, $y_{ij} \leq z_j$, and $y_{ij} \geq z_i + z_j - 1$, which force $y_{ij} = z_i z_j$ at any binary assignment. Both models are small enough at our test scales for CBC to prove optimality outright, and we count a solve as valid only when CBC certifies it; otherwise the instance is skipped rather than trusted.

One subtlety makes the first model exact despite its continuous flows. A single-commodity flow formulation that ships one unit of demand from a root to each required vertex has, for integer demands, an *integral* polytope: every extreme point of its feasible region assigns integer flow values. Intuitively, the flow polytope is the intersection of conservation equations whose constraint matrix is totally unimodular, so relaxing integrality of the flows changes nothing about the optimum. We may therefore declare flow variables continuous, which keeps the model small and lets CBC solve it in milliseconds, while losing no exactness. We verified this empirically as well: the MILP optimum coincides with brute-force enumeration of all feasible edge subsets on every small instance we tested.

2.5 Message-Passing Graph Neural Networks

A graph neural network (GNN) computes vector representations of vertices by repeatedly mixing each vertex's features with those of its neighbors. One *message-passing round* for vertex i with feature vector $x_i \in \mathbb{R}^d$ on a graph with weighted adjacency $A = (a_{ij})$ is

$$h_i \;=\; \sigma\Big(W_{\text{self}}\, x_i \;+\; \sum_{j \in \mathcal{N}(i)} a_{ij}\, W_{\text{nbr}}\, x_j\Big), \tag{3}$$

where W_{self} and W_{nbr} are learned or hand-set weight matrices, $\mathcal{N}(i)$ is the neighbor set of i, and σ is the elementwise sigmoid $\sigma(t) = 1/(1+e^{-t})$, which squashes its input into $(0, 1)$. The hidden state h_i now carries information about the one-hop neighborhood. Stacking more rounds propagates information farther, at the price of smoothing and compute; this project uses a single round, which is cheap enough for a low-power onboard computer and suffices because the pruning stage has already removed most long-range redundancy from the graph. A final *readout* maps hidden states to per-vertex scores that rank rovers for task assignment. Figure 3 sketches one round.

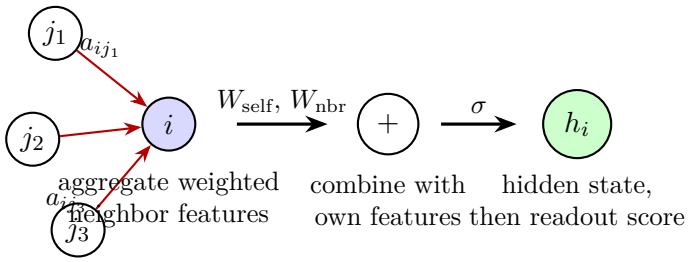


Figure 3: One message-passing round. Vertex i collects the feature vectors of its neighbors on the pruned graph, weighted by link strengths a_{ij}, combines them with its own features, applies the sigmoid nonlinearity, and obtains its hidden state h_i, from which a readout produces the rover score.

In our setting the input features are trust, battery level, available compute, and capability, and the adjacency weights come from the pruned topology. Two properties of this design matter later. First, the model is purely feed-forward arithmetic on small matrices, so it runs in microseconds per rover and imposes essentially no load on the fog tier. Second, because scoring happens *after* pruning, the GNN's view of the world is exactly the communication graph the mission will actually use, which ties allocation quality causally to pruning quality.

2.6 Trust Models for Multi-Agent Systems

A rover that has failed silently, or whose radio is degrading, should receive fewer critical tasks, even if it looks attractive on paper. Trust models make this notion quantitative. Following the trust literature, notably the survey of Sabater and Sierra (2005), we treat trust as a reputation score built from direct interaction outcomes, updated incrementally, and allowed to age.

Our model is an exponentially weighted moving average (EWMA). Each rover i carries a scalar trust $\tau_i \in [0, 1]$. When rover i is observed succeeding ($o = 1$) or failing ($o = 0$) on a task, its trust updates as

$$\tau_i \leftarrow (1 - \alpha)\, \tau_i + \alpha\, o, \tag{4}$$

with learning rate $\alpha \in (0, 1)$. Recent evidence dominates: a single bad outcome lowers trust immediately, while recovery requires a run of successes, which is the asymmetry we want for mission-critical assignment. Complementing the update is *aging*: trust that has not been refreshed by a recent observation relaxes toward a neutral prior, so stale evidence counts for less and a long-unobserved rover is neither trusted nor distrusted outright. Trust feeds the pipeline in two places, as a vertex feature for the GNN and as an edge feature for the pruning stage, so reputation influences both which links survive and which rovers are chosen.

2.7 Statistical Tools Used Throughout the Report

Because every experiment involves random seeds, hidden reliability draws, and stochastic solvers, single-run numbers mean little. We rely on four tools.

Paired bootstrap confidence intervals. To compare two methods run on the same seeds, we form the per-seed difference in the metric of interest, resample the differences with replacement 10,000 times, and take the 2.5 and 97.5 percentiles of the resampled means as a 95% confidence interval. Pairing on seeds removes instance-difficulty variance and sharpens the comparison.

Permutation tests. For a p-value we use an exact-style permutation test on the paired differences: under the null hypothesis that the two methods are exchangeable, the sign of each difference is arbitrary, so we flip signs at random across 20,000 permutations and measure how often the permuted mean is at least as extreme as the observed one. This makes no normality assumption.

Spearman rank correlation. When we ask whether a predicted score orders rovers the way realized outcomes do, we use Spearman's rank correlation, which compares orderings rather than values and is therefore robust to the arbitrary scale of the scores.

Normalized optimality gap. To grade a sampler against exact ground truth $H^\star$, we report

$$\text{gap} \;=\; \frac{H_{\text{SA}} - H^\star}{\max\bigl(1,\ |H^\star|\bigr)}, \tag{5}$$

which is nonnegative since $H^\star$ is minimal, equals zero exactly when the sampler is optimal, and is protected from division blow-ups when the optimal energy is near zero.

2.8 Notation

Table 2 summarizes the symbols used in this chapter and throughout the report.

Table 2: Notation used in this chapter and throughout the report.

Symbol	Meaning
$G_t = (V, E_t)$	dynamic rover graph at cycle t
$z \in \{0,1\}^n$	binary decision vector (one bit per optional edge)
Q, c	QUBO matrix and constant offset in $E(z) = z^\top Q z + c$
P	penalty weight for a hard-constraint violation
T_k, r	annealing temperature at sweep k and cooling rate
ΔE	energy change of a proposed move
H_{SA}, $H^\star$	sampler energy and exact optimal energy
x_i, h_i	input features and hidden state of vertex i
a_{ij}	weight of the edge between vertices i and j
W_{self}, W_{nbr}	GNN weight matrices for self and neighbor terms
σ	sigmoid nonlinearity, $\sigma(t) = 1/(1+e^{-t})$
τ_i	EWMA trust score of rover i
α	trust learning rate
d_i	degree limit of rover i

With these foundations in place, the next chapter makes the problem precise: given G_t and the current trust state, choose the subgraph, the rover scores, and the task assignment that maximize expected mission success under degree, budget, and connectivity constraints.

3 System Architecture

This chapter describes how the complete system is organised: the three physical tiers on which computation runs, the data flow within and across those tiers, the inventory of software components, and the design rules that shaped every implementation decision. The architecture is deliberately asymmetric. Almost all intelligence lives on one small computer that is physically co-located with the rover fleet, while the rovers themselves stay minimal and the cloud is used only where its latency does not matter.

3.1 The Three-Tier Deployment Model

We adopt the mist/fog/cloud hierarchy that is standard in edge computing for robotics, and map our pipeline onto it as follows.

Mist tier: the rovers. Each rover carries an ESP-NOW radio and a microcontroller with limited memory and no floating-point acceleration to speak of. The rover's responsibilities are confined to sensing (link quality to neighbours, battery state, odometry), executing the motion commands implied by an assigned task, and reporting task outcomes back to the fog node. No optimisation, no graph reasoning, and no learning happens on a rover. This choice reflects the hardware we target: a rover that must be cheap enough to lose in the field cannot host an annealer or a neural network. ESP-NOW provides short-range, low-power, peer-to-peer frames without the overhead of an association-based protocol, which suits the dense, rapidly changing local topologies we study.

Fog tier: the fleet brain. A single Raspberry Pi class computer sits within radio range of the fleet and runs the entire per-cycle decision loop: it assembles the candidate graph from rover reports, formulates and solves the constrained QUBO edge pruning problem, scores rovers with the trust-aware GNN on the pruned graph, and performs the greedy capacity-aware task assignment. This tier is the reason for the most consequential engineering constraint of the project: *all per-cycle numerical code is pure NumPy over tiny matrices.* A cycle must complete in well under the control period on a few-watt ARM processor, so there is no room for a deep learning framework, a sparse linear algebra stack, or a commercial solver. Every matrix in the inference path is small enough that dense NumPy with a handful of temporaries is both the simplest and the fastest honest option. The message-passing round of the GNN is a single adjacency product followed by an elementwise nonlinearity, and the projected edge annealer manipulates flat bit vectors and small dictionaries rather than any external optimisation library.

Cloud tier: training, amortised solves, analytics. Anything whose latency does not couple into the control loop is pushed to the cloud tier. Three workloads live there. First, offline training of the GNN readout layer on logged outcomes; we report in Chapter 6 that this training did not beat the fixed readout on held-out data, but the infrastructure remains part of the architecture. Second, amortised heavy solves: the pruning QUBO can be re-solved every k cycles rather than every cycle, and heavy verification solves (the exact MILP baseline, or cross-checks of annealer output) are cloud jobs, not field jobs. Third, fleet-level analytics over mission logs.

Figure 4 summarises the deployment and the data that crosses tier boundaries.

3.2 Per-Cycle Data Flow

One control cycle proceeds through six stages, which we describe in order, noting what crosses a tier boundary at each step.

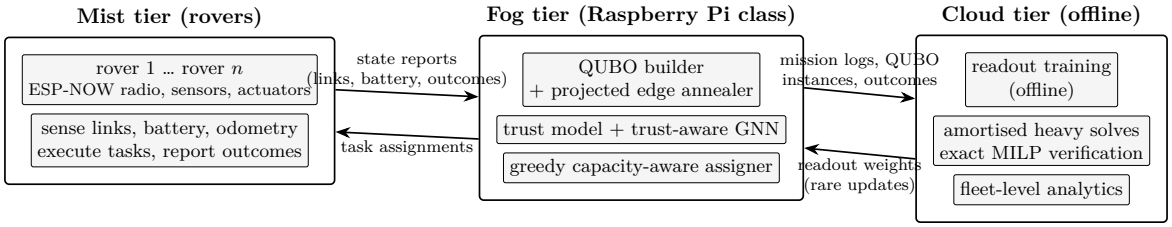


Figure 4: Three-tier deployment. The mist tier (rovers) senses and acts; the fog tier runs the complete per-cycle decision loop on a Raspberry Pi class computer using pure NumPy; the cloud tier performs offline training, amortised or verification-grade solves, and fleet analytics.

Stage 1: graph assembly (mist to fog). Each rover reports its current estimate of link quality to every neighbour it can hear, its battery level, its compute availability, and its position estimate. These reports travel over ESP-NOW to the fog node, which merges them with its own per-node records (trust, freshness of last contact, task criticality, processing cost) to build the full dynamic candidate graph: every admissible rover pair is an edge, and every edge carries a feature vector in $[0, 1]^7$ covering trust, link quality, battery, proximity, freshness, task criticality, and processing cost.

Stage 2: constrained QUBO pruning (fog). The candidate graph is denser than the GNN needs, and dense graphs dilute the trust signal and increase inference cost. The QUBO builder encodes edge selection as a binary quadratic program: a linear utility per edge (the learned-weight combination of the seven features), soft penalties for graph size and for churn against the previous cycle's decisions, and hard constraints with dominant penalty weights that enforce per-node degree limits, a global processing budget, and single-commodity flow connectivity from a root to every required node. The production solver, the projected edge annealer, moves directly in the space of feasible edge sets: it toggles or swaps edges and repairs each candidate via an exact slack-and-flow recomputation, so infeasible states are never committed. The output is a pruned subgraph plus, for auditability, the raw solution vector and a list of any violated constraints.

Stage 3: trust-aware scoring (fog). The trust model maintains an exponentially weighted moving average of per-node reliability, with ageing so that stale evidence decays. The GNN takes the pruned adjacency, per-node features (trust, battery, available compute, capability), and performs one message-passing round with a sigmoid nonlinearity, then a linear readout scores each rover against the current task requirements. All of this is dense NumPy on matrices of order $n \times n$ with n the fleet size, comfortably inside the fog compute budget.

Stage 4: assignment (fog to mist). A greedy capacity-aware matcher assigns tasks to the highest-scoring rovers that still have free capacity. The assignments are radioed down to the rovers, which execute them.

Stage 5: outcome sampling (mist). Task outcomes are realised on the rovers. In simulation, outcomes are sampled from a hidden, spatially correlated reliability field, and the probability of successful delivery depends causally on the pruned topology through the paths the assignment actually uses; this design, introduced after reviewer pressure, prevents the evaluation from being merely correlational.

Stage 6: trust update (fog, then feedback). Reported outcomes update the trust model, closing the loop: the updated trust values are inputs to the edge features of the next cycle. Mission logs, QUBO instances, and outcome traces are streamed or batched to the cloud tier

for the offline workloads described above; the only cloud-to-fog traffic in normal operation is an occasional readout-weight update.

3.3 Component Inventory

Table 3 lists the components of the system by role. Two deliberate asymmetries are worth flagging. First, two annealers exist: a bit-level simulated annealer that manipulates raw bit strings and is used *only* as a benchmark subject, and the projected edge annealer that is the production solver. The benchmark exists precisely because the bit-level solver performs poorly (it cannot cross penalty barriers and returned infeasible states on most benchmark instances), which is itself one of our audit findings. Second, the exact MILP baseline (CBC via PuLP) is faster and exact on our instance sizes, which we report honestly as a negative result for QUBO on classical hardware; it serves as the ground truth oracle for the solver audit.

Table 3: Component inventory. "Tier" indicates where the component runs in deployment; benchmark and verification components are cloud-side or development-side tools.

Component	Role	Tier	Technology
QUBO builder	constrained edge-pruning encoding	fog	NumPy
Bit-level annealer	benchmark subject only	cloud/dev	NumPy
Projected edge annealer	production QUBO solver	fog	NumPy
Trust model	EWMA reliability with ageing	fog	NumPy
Trust-aware GNN	rover scoring on pruned graph	fog	NumPy
Greedy assigner	capacity-aware task matching	fog	NumPy
Exact MILP baseline	ground-truth oracle, audit	cloud	CBC via PuLP
Scaling runner	fleet-size experiments	cloud	NumPy
Hostile test suite	491 checks, sections A to N	dev	Python stdlib
Causal simulation harness	end-to-end evaluation, statistics	cloud	NumPy
Physics integration	realistic rover dynamics	cloud	Gazebo, ROS 2, Docker
Fleet console	live visualisation and tuning	dev	three.js, React, Python

The **causal simulation harness** drives the whole pipeline end-to-end against the hidden reliability field and hosts the pruning baselines (no pruning, threshold pruning, top-k pruning, full graph) and assignment baselines (trust-only, no-graph, random) used in the ablation studies. It reports paired bootstrap confidence intervals and permutation p-values so that every comparative claim carries an uncertainty statement.

The **physics integration** replaces the abstract kinematics of the simulator with Gazebo Harmonic under ROS 2 Jazzy, packaged in Docker so that the planner node runs unmodified against simulated physics; this validates that the fog-side loop tolerates realistic sensing noise and actuation latency.

The **fleet console** is a browser application: a custom three.js rendering engine (instanced meshes, a single instanced shader draw for link arcs) with a React interface and Zustand state management, served by a Python standard-library HTTP backend that steps live simulation sessions and exposes in-place parameter tuning. It exists for demonstration and for interactive exploration of solver parameters, and its comparison menu names the baselines that beat our pipeline rather than hiding them.

The **hostile test suite** deserves special mention as an architectural component rather than an afterthought. Its 491 checks, organised into sections A to N, include an independent re-derivation of the QUBO energy that is cross-verified against the quadratic form itself, brute-force enumeration of small instances as ground truth, exact MILP cross-checks on larger instances, and sensitivity sweeps over annealer hyperparameters and penalty scales.

3.4 Design Rules

Four rules, adopted early and enforced throughout, explain most of the concrete choices above.

Rule 1: the fog compute budget is real. Every per-cycle component must run in milliseconds on a Raspberry Pi class machine. This rules out deep learning frameworks (hence a GNN that is one dense message-passing round in NumPy), rules out heavy sparse or optimisation libraries in the inference path (hence a hand-rolled projected annealer), and motivates QUBO pruning in the first place: the pruned graph is cheaper to score, so the pruner partially pays for itself.

Rule 2: no heavy dependencies anywhere. The numerical stack depends on NumPy alone. The exact baseline uses PuLP with CBC, both permissively licensed and installable without system libraries; notably, we avoid SciPy entirely. The web backend uses only the Python standard library. The physics simulator is containerised with Docker precisely so that its heavier ROS 2 dependency chain is isolated from the numerical core.

Rule 3: every claim is paired with a test. The 491-check hostile suite is co-developed with the code, not appended to it. The solver exactness rates quoted in later chapters are enforced by floor checks in the suite, the simulation statistics are regression checked, and new components are expected to arrive with their own hostile tests.

Rule 4: negative results are kept. The architecture is an audit, not only a proposal, and the system is built to surface unfavourable comparisons rather than suppress them. The exact MILP baseline that dominates the QUBO solver on classical hardware is a first-class component; the readout-training null result is reported with its p-value; the pruner comparison in which threshold pruning wins on raw mission success at the ten-rover scale appears in the results with its significance test. We regard these findings as outputs of the system, on equal footing with its successes.

3.5 Summary

The system places a complete decision loop on one modest fog computer by discipline rather than by excess compute: a minimal mist tier, a NumPy-only fog tier running pruning, scoring, and assignment, and a cloud tier reserved for training, verification-grade solves, and analytics. The surrounding tooling, from the MILP oracle to the hostile suite to the fleet console, exists to make the central claims falsifiable and the negative results impossible to misplace.

4 The Constrained QUBO Formulation

This chapter presents the mathematical core of our pipeline: the reduction of dynamic edge pruning to a constrained quadratic unconstrained binary optimisation (QUBO) problem. Each cycle of the simulation begins with a candidate communication graph that may contain far more links than the rovers can actively maintain. We wish to keep a small, high-quality, connected subgraph on which the trust-aware inference of the following chapters then operates. The QUBO formalism is attractive for this task because it accepts both linear utility terms and pairwise interaction terms natively, because it is the input format of quantum and quantum-inspired annealing hardware, and because it forces us to state every modelling decision as an explicit, auditable term in a single energy function. The price we pay is that all hard constraints must be expressed as penalty terms, and Section 4.4 shows how we make those penalties provably dominant.

Table 4: Per-edge features, all normalised to $[0, 1]$.

Feature	Symbol	Meaning
Trust	f_e^{tr}	EWMA trust of the endpoint pair, learned from past outcomes.
Link quality	f_e^{lq}	Recent packet-delivery success on this link.
Battery	f_e^{ba}	Minimum battery level of the two endpoints.
Proximity	f_e^{pr}	Normalised closeness, decreasing in Euclidean distance.
Freshness	f_e^{fr}	Recency of the last successful message on this link.
Task criticality	f_e^{tc}	Whether the link serves a mission-critical flow.
Processing cost	f_e^{pc}	Compute required to maintain and monitor the link.

4.1 Candidate edges and features

Let $V = \{1, \ldots, n\}$ be the set of rovers and let $E_{\mathrm{cand}} \subseteq V \times V$ be the set of candidate edges, that is, pairs of rovers currently within communication range. Every candidate edge $e = (i, j) \in E_{\mathrm{cand}}$ is described by a feature vector $\mathbf{f}_e \in [0, 1]^7$ whose components are listed in Table 4. All features are normalised to the unit interval before the formulation module is invoked; any value outside $[0, 1]$ is rejected at construction time (Section 4.5).

The first six features describe desirable properties; the seventh is a cost. We aggregate the desirable features into a scalar edge utility

$$U_e \;=\; w_{\mathrm{tr}} f_e^{\mathrm{tr}} + w_{\mathrm{lq}} f_e^{\mathrm{lq}} + w_{\mathrm{ba}} f_e^{\mathrm{ba}} + w_{\mathrm{pr}} f_e^{\mathrm{pr}} + w_{\mathrm{fr}} f_e^{\mathrm{fr}} + w_{\mathrm{tc}} f_e^{\mathrm{tc}}, \tag{6}$$

with weights

$$(w_{\mathrm{tr}}, w_{\mathrm{lq}}, w_{\mathrm{ba}}, w_{\mathrm{pr}}, w_{\mathrm{fr}}, w_{\mathrm{tc}}) = (0.25, 0.25, 0.10, 0.10, 0.15, 0.15). \tag{7}$$

The weights are validated to sum to one, so $U_e \in [0, 1]$ and can be read as the fraction of an ideal link that edge e realises. Trust and link quality dominate because they carry the causal signal for delivery success; battery and proximity act as mild tie breakers; freshness and task criticality sit between. Because every feature lies in $[0, 1]$, no single edge is ever worth more than about one unit of soft energy. This fact is the basis of the penalty argument below.

4.2 Decision variables

Not every edge is subject to optimisation. A subset $E_{\mathrm{hard}} \subseteq E_{\mathrm{cand}}$ of edges may be marked *hard-critical* by the mission layer, for example because a link is the sole path to a stranded rover. Hard-critical edges are always kept and are excluded from the variable set entirely; their utility and their contribution to degree and budget accounting enter the energy only as constants. For each remaining optional edge $e \in E_{\mathrm{opt}} = E_{\mathrm{cand}} \setminus E_{\mathrm{hard}}$ we introduce one binary variable

$$x_e = \begin{cases} 1, & \text{if edge } e \text{ is kept,} \\ 0, & \text{if edge } e \text{ is pruned.} \end{cases} \tag{8}$$

The full variable vector $\mathbf{x} \in \{0, 1\}^{|E_{\mathrm{opt}}|}$ is the primary output of the solver; slack and flow variables introduced below are auxiliary and are projected away when the solution is extracted.

Table 5: Soft coefficients and hard-penalty magnitudes. Hard penalties dominate the soft objective by roughly two orders of magnitude.

Term	Role	Value
Edge penalty λ	soft, prefers fewer links	0.08
Stability penalty μ	soft, discourages churn	0.15
Degree penalty	hard, per-node degree caps	100
Budget penalty	hard, global processing budget	100
Flow penalty	hard, connectivity to the root	500
Flow capacity penalty	hard, flow only on kept edges	500
Cost scale s	budget quantisation factor	10

4.3 Soft objective

The soft part of the energy encodes two desiderata. First, we want to keep edges of high utility while preferring a sparse graph overall. Second, because the formulation runs every cycle in a live network, we want to discourage gratuitous churn: pruning and re-adding the same edge in alternating cycles costs signalling overhead and destabilises the trust estimates that depend on link history.

Let $x_e^{\text{prev}} \in \{0,1\}$ denote the decision taken for edge e in the previous cycle (zero for edges seen for the first time). The soft energy is

$$E_{\text{soft}}(\mathbf{x}) \;=\; \sum_{e \in E_{\text{opt}}} \big(-U_e + \lambda\big) x_e \;+\; \mu \sum_{e \in E_{\text{opt}}} \big|x_e - x_e^{\text{prev}}\big|, \tag{9}$$

with edge penalty $\lambda = 0.08$ and stability penalty $\mu = 0.15$. The linear term keeps exactly those edges with $U_e > \lambda$: keeping an edge whose utility clears the small threshold lowers the energy, and keeping a worthless edge raises it. The second sum counts the number of flips relative to the previous decision. Since x_e and x_e^{prev} are binary, $|x_e - x_e^{\text{prev}}|$ is linear in x_e once the constant x_e^{prev} is fixed, so (9) is a purely linear function of $\mathbf{x}$. The quadratic structure of the problem enters only through the constraints, to which we now turn.

4.4 Hard constraints as dominant penalties

A QUBO has no explicit constraints, so each hard requirement is encoded as a non-negative penalty term that vanishes exactly on feasible solutions. For the encoding to be correct, breaking a constraint must always cost more than any possible soft gain. Inspection of (9) shows that the soft energy contributed by a single edge is bounded in magnitude by $1 + \lambda + \mu < 1.3$. We therefore choose penalty magnitudes of 100 and 500: any single violation costs at least 100 while the entire soft objective over a realistic instance is worth at most a few units, so no feasible-gain trade can ever justify an infeasible configuration. Table 5 summarises the coefficients.

Degree caps. Each rover i has a degree limit $d_i^{\max}$ reflecting its radio and compute capacity. Let $\delta(i)$ be the set of optional edges incident to i and let k_i be the number of hard-critical edges at i. The constraint $k_i + \sum_{e \in \delta(i)} x_e \le d_i^{\max}$ is encoded with a non-negative integer slack $s_i \ge 0$ satisfying

$$\sum_{e \in \delta(i)} x_e + s_i \;=\; d_i^{\max} - k_i, \qquad P_{\text{deg}} \;=\; 100 \sum_{i \in V} \Big(\sum_{e \in \delta(i)} x_e + s_i - (d_i^{\max} - k_i) \Big)^2. \tag{10}$$

The integer slack is represented in binary as $s_i = \sum_{b=0}^{B_i - 1} 2^b y_{i,b}$ with $B_i = \lceil \log_2(d_i^{\max} - k_i + 1) \rceil$ slack bits $y_{i,b} \in \{0,1\}$. Squaring the residual produces pairwise products $x_e x_{e'}$, $x_e y_{i,b}$, and $y_{i,b} y_{i,b'}$, which is exactly the quadratic form a QUBO accepts. The penalty vanishes only when the degree constraint holds with the correct slack.

Global processing budget. Maintaining a link consumes processing resources, and the fleet shares a global budget $B^{\max}$ expressed in integer cost units. Because QUBO penalties require exact arithmetic, real-valued processing costs are quantised: the cost of edge e is $c_e = \lceil s \cdot f_e^{\text{pc}} \rceil$ with cost scale $s = 10$, so costs are integers between 0 and 10. With c_{hard} the total cost of hard-critical edges and an integer slack $t = \sum_b 2^b z_b \geq 0$ built from binary slack bits z_b, the budget constraint becomes

$$P_{\text{bud}} \;=\; 100\Big(\sum_{e \in E_{\text{opt}}} c_e x_e + t - (B^{\max} - c_{\text{hard}})\Big)^2. \tag{11}$$

Quantisation conservatively overestimates each cost, so a solution feasible under the QUBO is always feasible under the true real-valued budget.

Connectivity by single-commodity flow. Sparsity and degree caps must not disconnect the network. We designate a root rover $r \in V$ and a set $V_{\text{req}} \subseteq V$ of required nodes (every rover that must remain reachable this cycle) and demand that every required node be connected to r through kept edges. Rather than one flow per destination, we use a single-commodity flow: the root injects $|V_{\text{req}}|$ units of flow into the network, each required node absorbs one unit, and flow may travel only along kept edges. For every undirected candidate edge $e = (i, j)$ we introduce two directed flow variables $f_{ij}, f_{ji} \in \{0, 1\}$ indicating whether an arc carries flow. Flow conservation at every node $i \neq r$ reads

$$\sum_j f_{ji} - \sum_j f_{ij} \;=\; \begin{cases} 1, & i \in V_{\text{req}} \setminus \{r\}, \\ 0, & \text{otherwise}, \end{cases} \tag{12}$$

and at the root the net outflow must equal $|V_{\text{req}}|$. Because the right-hand sides are integers, the standard integrality property of single-commodity flow guarantees that binary flow variables suffice: if any feasible flow exists, an integral one exists. This is why a single commodity is enough here. We do not ask *how much* traffic each required node receives, only whether it is reachable, and unit demands with one source make the reachability question a single-commodity problem rather than a multi-commodity one, keeping the variable count linear in the number of arcs.

Each constraint of (12) is penalised quadratically with weight 500:

$$P_{\text{flow}} \;=\; 500 \sum_{i \in V} \Big(\text{inflow}_i - \text{outflow}_i - d_i\Big)^2, \tag{13}$$

where d_i is the demand defined by (12). Flow must also be forbidden on pruned edges. Since demands are unit and the total flow is $|V_{\text{req}}|$, at most $|V_{\text{req}}|$ arcs can carry flow, so a capacity slack encoding links flow to edge variables: an arc may carry flow only if its edge is kept, enforced by a second family of quadratic penalties with weight 500 and per-edge integer capacity slack bits. A required node that cannot be reached leaves its conservation residual at 1, firing a penalty of 500 and rendering the configuration strictly worse than any feasible one.

The complete energy is

$$E(\mathbf{x}, \mathbf{y}, \mathbf{z}, \mathbf{f}) \;=\; E_{\text{soft}}(\mathbf{x}) + P_{\text{deg}} + P_{\text{bud}} + P_{\text{flow}} + P_{\text{cap}}, \tag{14}$$

which the formulation module expands into an explicit upper-triangular matrix Q such that $E = \mathbf{v}^\top Q \mathbf{v} + \text{const}$ over the concatenated variable vector $\mathbf{v}$. In summary, the variable structure comprises: one binary per optional edge; $\Theta(\log d_i^{\max})$ slack bits per node for degree caps and $\Theta(\log B^{\max})$ bits for the budget; two flow binaries per undirected edge, one per directed arc; and capacity slack bits per edge.

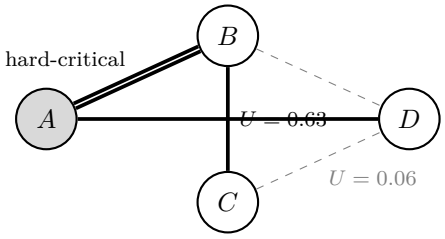


Figure 5: Toy instance with four rovers and five candidate edges. Solid lines are kept edges, the double line is hard-critical, and dashed grey lines are pruned. Rover A is the flow root.

4.5 Input validation at construction

The formulation module validates its inputs hard and refuses to build an ill-posed instance rather than silently producing a meaningless QUBO. Three families of errors raise immediately. First, feature vectors with any component outside $[0, 1]$ are rejected, as are weight vectors that do not sum to one; the normalisation invariants of Table 4 are preconditions, not suggestions. Second, any required node that is unreachable from the root in the full candidate graph E_{cand} is rejected: no feasible pruning exists, so the instance is unsatisfiable by construction and the caller must either relax the requirement or extend the communication range. Third, a global budget below the cost of the cheapest spanning tree connecting the required nodes is rejected for the same reason. Failing fast at the formulation boundary is what allows the downstream solver audit of the next section to attribute every infeasible sample to the solver rather than to the encoding.

4.6 A worked toy example

We illustrate the construction on a four-rover instance, drawn in Figure 5. Rovers A (the root), B, C, and D admit five candidate edges; the edge (A, B) is marked hard-critical. Consider edge (B, C) with feature vector

$$\mathbf{f}_{BC} = (f^{\text{tr}}, f^{\text{lq}}, f^{\text{ba}}, f^{\text{pr}}, f^{\text{fr}}, f^{\text{tc}}) = (0.9,\ 0.8,\ 0.6,\ 0.7,\ 0.5,\ 0.0).$$

Its utility by (6) is

$$U_{BC} = 0.25(0.9) + 0.25(0.8) + 0.10(0.6) + 0.10(0.7) + 0.15(0.5) + 0.15(0.0) = 0.63,$$

so the linear coefficient in (9) is $-0.63 + 0.08 = -0.55$: keeping (B, C) lowers the energy and is soft-preferred. The edge (C, D), by contrast, carries stale trust and poor link quality, giving $U_{CD} = 0.06$ and a coefficient of $+0.02$: it is kept only if constraints demand it.

Suppose the degree cap of rover C is $d_C^{\max} = 2$ and the budget is tight. The optimum then keeps the chain (A, B), (B, C), (C, D) if the budget permits, because connectivity of the required node D is worth 500 penalty units while the edge (C, D) costs only 0.02 in soft energy and a few quantised budget units. If the budget cannot cover the chain, the solver routes flow through (A, D) instead and prunes (C, D). The qualitative pattern is representative of larger instances: the soft objective shapes the graph around high-trust, high-quality links, while the penalty terms silently guarantee that every required rover stays attached to the root. The example is illustrative; Section 4.7 describes how we verify that the energy function evaluates such trades correctly.

4.7 Verification of the energy function

Every downstream conclusion in this report depends on the Q matrix encoding the intended energy. We therefore verify the construction at three independent levels. First, the energy function shipped in the formulation module is cross-checked against an independent re-derivation written from equations (9) to (14) directly over selected edge sets, bypassing the matrix entirely; the two agree on randomly sampled configurations to within 10^{-7}, the scale of floating-point accumulation.

Second, on small instances we enumerate all $2^{|E_{\text{opt}}|}$ edge subsets exhaustively, evaluate each with the independent energy, and confirm that the argmin over the matrix representation matches the enumeration exactly. Third, we solve the QUBO exactly by a standard linearisation, introducing one auxiliary variable per nonzero quadratic term, with an exact integer programming solver; the certified optimum again agrees with enumeration. The three routes, analytic re-derivation, brute force, and exact linearised solve, share no code, so agreement to 10^{-7} rules out a systematic encoding error. The solver benchmark of later chapters, which compares annealing heuristics against this ground truth, would be meaningless without it.

4.8 Summary

We have reduced dynamic edge pruning to a QUBO whose soft terms express utility, sparsity, and temporal stability, and whose hard terms, degree caps, a quantised global budget, and single-commodity flow connectivity, are encoded with penalties two orders of magnitude larger than any possible soft gain. The construction validates its inputs at the boundary, produces an explicit quadratic matrix suitable for both classical and annealing solvers, and is verified against independent ground truth. The next chapter describes the solver that operates on this encoding and the audit that establishes how close it comes to the exact optimum.

5 Solvers: from Bit Flips to Feasibility Projection

The QUBO formulation of the previous chapter defines a discrete energy landscape over binary vectors, one bit per optional edge plus auxiliary slack and flow variables. This chapter asks how that landscape can be searched, and why the answer is less obvious than the phrase "run an annealer" suggests. We proceed in three steps. First we describe a conventional bit-level simulated annealer and the samplers against which it is benchmarked. Second, we show that this entire family of methods performs poorly on penalty-encoded constrained instances, and we trace the failure to a structural property of the landscape rather than to parameter tuning. Third, we describe the production solver, a projected annealer whose moves live inside the feasible set, and we report its measured accuracy against exact ground truth.

5.1 A bit-level simulated annealer

Our reference annealer operates directly on the raw bit string. Each variable of the QUBO, including edge bits, integer slack bits, and directed flow bits, is a binary decision, and a move flips one of them. A flip changes the energy by an amount computed incrementally from the Q matrix, so a single move costs $O(n)$ in the number of variables rather than a full quadratic energy re-evaluation. The acceptance rule is the standard Metropolis criterion: downhill moves are taken unconditionally, and uphill moves of magnitude ΔE are accepted with probability $\exp(-\Delta E/T)$.

The schedule is geometric cooling. Starting from an initial temperature $T_0 = 10$, the temperature is multiplied by a cooling rate of 0.98 after every two sweeps, where one sweep attempts one move per variable on average, until a final temperature of 10^{-3} is reached. Two extensions round out the design. First, with a configurable pair probability, a move flips two bits simultaneously instead of one; this helps the sampler traverse configurations in which two compensating changes are neutral jointly but costly individually. Second, the annealer accepts warm initial states, so a previously discovered configuration can seed a new run rather than starting from a random string.

Because the word "effort" is otherwise easy to game, the annealer maintains an explicit counter of energy evaluations. Every comparison in this chapter reports that counter, so that a solver which wins by doing more work is visibly doing more work. This habit of honest accounting matters here, because the production solver of Section 5.4 does more work per move and we want that visible rather than hidden.

Table 6: Schedules and defaults of the solvers compared in this chapter. The bit-level annealer, `neal`, tabu, and simulated quantum annealing (SQA) all operate on the raw bit string of the encoded QUBO. The projected annealer operates on feasible edge sets; its temperature range is lower because its moves change the soft objective only.

Solver	T_{init}	T_{final}	Cooling	Steps per T	Restarts
Bit-level SA	10	10^{-3}	0.98	2 sweeps	1 (warm optional)
`neal`	default `beta_range`			default sweeps	1
Tabu search	n/a (tenure-based)			default iterations	1
SQA (OpenJij)	default transverse-field schedule			default sweeps	1
Projected annealer	1.0	10^{-2}	0.998	4	5

5.2 Why bit-level annealing fails on constrained QUBOs

The failure mode is easy to state and hard to fix within the bit-flip paradigm. In our encoding, hard constraints are enforced by penalty terms whose weights, between 100 and 500, are chosen to dominate the soft objective: the utility of a single edge is at most about 1, so breaking one constraint costs at least one hundred times more than any single edge can ever be worth. The intended consequence is that any optimizer which minimizes total energy will respect the constraints. The unintended consequence is that the energy landscape becomes a set of deep, narrow valleys separated by enormous barriers.

Consider a Metropolis sampler sitting in a feasible configuration and contemplating a single bit flip. Almost every flip that touches a constraint variable produces a state that violates a constraint, with an energy increase of at least 100. At temperature $T = 1$, such a move is accepted with probability below e^{-100}, which is zero for all practical purposes. The sampler is therefore confined to the feasible region it started in, able to leave it only at the high temperatures of the early schedule, when acceptance is indiscriminate and no useful signal is retained. Cooling then strands it in whichever valley it happened to occupy. Random restarts do not rescue the method: starting from uniformly random bit strings, the bit-level annealer returned an infeasible final state in 19 of 25 test instances. The sampler could not even find its way back into the feasible region, let alone optimize within it.

We emphasize that this is not a complaint about a particular implementation. Penalty-encoded constraints create, by construction, barriers that single-bit moves cannot cross at useful temperatures. Stronger cooling schedules, more sweeps, or pair moves shift the numbers by a few percentage points but do not address the geometry.

5.3 Sampler benchmark on the real Q matrices

To quantify the claim, we benchmarked four samplers on the actual Q matrices produced by our encoding for 25 representative instances, with ground truth defined two ways. For each instance, the exact optimum of the constrained problem was obtained by complete enumeration of the optional-edge subsets, and the exact minimum of the encoded QUBO was obtained by a linearized exact solve with CBC, which converts each quadratic term into an auxiliary variable and proves optimality with a branch-and-bound search. The two agree wherever both are computable; the CBC solve is the ground truth for what a sampler *should* return.

Table 7 summarizes the outcome. The local bit-level annealer reaches the exact optimum in 56% of instances. D-Wave's `neal` sampler, a mature open-source simulated annealing implementation, reaches 60%. Tabu search reaches 64%, the best of the four, and simulated quantum annealing through OpenJij reaches 48%, the worst. No sampler clears two thirds. Figure 6 shows the same numbers against the exact reference at 100%.

Two conclusions follow. First, the weakness is shared across implementations and paradigms: classical annealing, tabu search, and simulated quantum annealing all struggle, which supports the geometric explanation of Section 5.2 over any implementation-specific one. Second, the Q

Table 7: Exact-solution rates on 25 instances with enumerated ground truth. "Infeasible" counts runs whose returned bit string violates at least one constraint. The projected annealer is excluded from this comparison by design; see Section 5.6.

Sampler	Exact rate	Returned infeasible
Bit-level SA (ours)	56%	19 of 25 (random restarts)
`neal`	60%	n/a
Tabu search	64%	n/a
SQA (OpenJij)	48%	n/a
Exact linearized solve (CBC)	100% (reference)	0

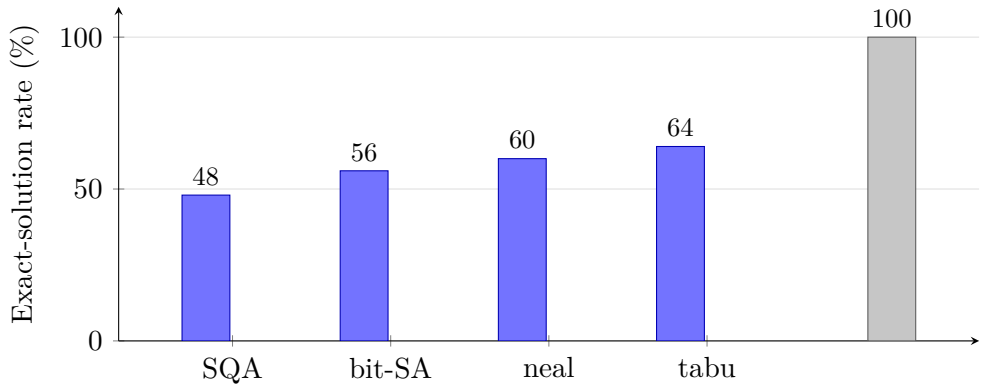


Figure 6: Exact-solution rates of four samplers on 25 QUBO instances derived from our encoding. The gray bar is the exact linearized CBC solve at 100%, which defines the ground truth against which all sampler rates are measured; it is a reference, not a competitor.

matrix itself is not at fault, because the exact linearized solve of that same matrix recovers the true optimum every time. The encoding is a faithful specification; generic samplers are simply poor at searching it.

5.4 The production solver: feasibility-projected edge annealing

The production solver changes the search space rather than the search algorithm. Instead of annealing over raw bit strings and hoping to land inside the feasible region, it anneals directly over the set of feasible edge subsets. A state is a set of kept edges together with the slack and flow assignments that certify its feasibility; the energy of a state is exactly the QUBO energy evaluated at that certificate. On the feasible manifold the penalty terms vanish identically, so the energy reduces to the soft objective, and the sampler never wastes a single evaluation on a constraint violation.

Two move types generate candidates, as illustrated in Figure 7. A *toggle* adds or removes one optional edge. A *swap* exchanges two edges, one in and one out, which allows the solver to reallocate budget or degree capacity between parts of the graph in a single step. Not every candidate is feasible: adding an edge can exceed a degree cap or the global budget, and removing one can disconnect a required node from the root. Each candidate is therefore passed through a projection step, an exact repair procedure that recomputes the integer slack variables and rebuilds a spanning-tree flow certifying connectivity. If the repair succeeds, the repaired state is the candidate actually scored; if no feasible repair exists, the move is rejected outright. Projection is deterministic and exact, so the solver's trajectory never leaves the feasible set.

Restarts require care, for the same geometric reason that defeats bit-level annealing. Greedy edge-by-edge growth, adding the highest-utility edge that keeps the partial solution feasible, cannot cross the infeasible prefix states that the connectivity constraint creates: a cheap subtree can be constructed that is locally feasible yet admits no feasible completion within budget. Restart

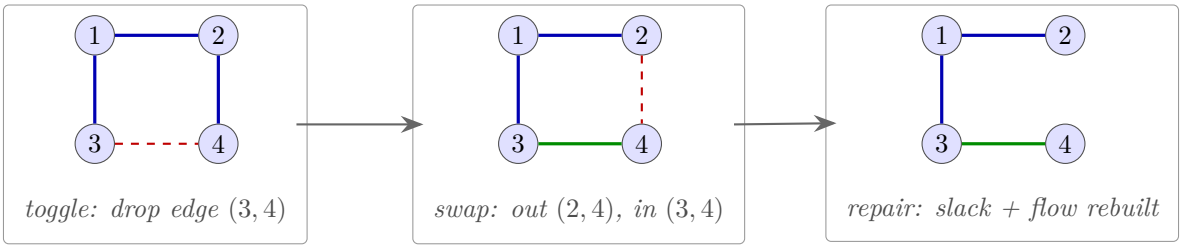


Figure 7: The move set of the projected annealer. Solid blue edges are kept, the dashed red edge is removed, and the green edge is added. A toggle changes one optional edge; a swap exchanges two. The right panel shows the projection step: after the move, integer slack variables and a spanning-tree flow from the root are recomputed exactly, and the candidate is accepted only if the repair succeeds.

states are therefore generated as degree-cap-aware random spanning trees, sampled so that every partial state is a tree that already respects the degree limits, and then completed with any budget-feasible extra edges. This guarantees that every restart begins inside the feasible manifold with a diverse geometry.

After cooling, a deterministic polish drives the incumbent to a local fixpoint. The polish alternates a pass of all single toggles with a pass of all pairwise swaps, accepting every strict improvement, until neither pass finds one. The polish is cheap relative to the anneal and removes the residual noise that a finite temperature schedule leaves behind.

It is worth being precise about what the QUBO has become in this design. The QUBO remains the specification of the problem: the energy function, the constraint weights, and the feasibility test are all defined by it, and the Q matrix is still exported for hardware backends, so a quantum or dedicated annealing device can be substituted for the classical sampler without touching the formulation. The projected annealer is best understood as a classical sampler that exploits the known structure of the penalty terms to restrict itself to the subspace where those terms are zero.

5.5 Defaults, measured cost, and accuracy

The production defaults are deliberately mild, because the feasible manifold is a friendlier landscape than the raw bit space. The initial temperature is 1.0, the final temperature is 10^{-2}, the cooling rate is 0.998, and each temperature holds for four steps. Toggles and swaps are proposed with equal probability 0.5. Five restarts are drawn from the random spanning-tree generator, and in the live system the previous cycle's kept edges are supplied as a warm start, so that successive cycles refine an incumbent rather than restarting from scratch.

Measured on the benchmark instances, a solve performs about 2.4×10^4 energy evaluations and takes about 2.1 seconds of wall time on commodity hardware. The accuracy numbers are the following.

- **Small instances.** On the 25 instances with enumerated ground truth, the projected annealer returns the exact optimum in 100% of cases, against 56% to 64% for the raw-space samplers of Table 7.
- **Larger instances.** On 20 larger instances where enumeration is infeasible and ground truth comes from the exact MILP solve of the original constrained problem, the projected annealer is exact in 95% of cases, with a mean true optimality gap of 7×10^{-5} and a worst-case gap of 1.4×10^{-3}.
- **Robustness.** Across a sensitivity sweep that perturbs the initial and final temperatures, the cooling rate, the restart count, the penalty scales, and the pair probability, the solver remains exact in 87% of configurations. Accuracy degrades gracefully rather than catastrophically when the schedule is mistuned.

5.6 An honest framing

The projected annealer is not entered into the sampler comparison of Section 5.3, and we want the reason on record. Its move set is richer than what a generic sampler is permitted: it exploits domain knowledge, namely the structure of the constraint set, the meaning of the slack variables, and the availability of an exact repair procedure, that a hardware annealer or a black-box sampler acting on a raw Q matrix does not have. Comparing it against `neal`, tabu, or OpenJij on equal footing would conflate two different questions: how well generic samplers search penalty-encoded QUBOs, and how well a structure-aware classical algorithm solves the underlying constrained problem. The first question is the scientifically interesting one for assessing quantum and quantum-inspired hardware, and it is answered by Table 7. The projected annealer is therefore reported as an engineering note: it is the solver the deployed system actually uses, and its near-exact results establish that the QUBO formulation itself, not any solver, is the binding constraint on end-to-end performance. Where a classical exact optimum is affordable, the MILP solve remains both faster and provably optimal, and the evaluation chapter makes that comparison directly.

6 Trust Modeling and the Trust-Aware GNN

This chapter describes the learning and inference components that sit on top of the pruned communication graph produced by the QUBO layer. The design centers on a per-node trust model that converts observed mission outcomes into a slowly evolving reputation signal, and a lightweight graph neural network that scores candidate rovers against task requirements using exactly the pruned topology as its input graph. We first state the deployment constraint that shapes every architectural choice, then describe the node features, the trust model, the message-passing layer, the assignment rule, and finally the empirical evidence: what the trust model tracks, what the GNN contributes, and where our training attempt produced an honest null result.

6.1 Deployment constraint: mist, fog, and cloud tiers

The target platform is a three-tier architecture. Rovers and their immediate wireless neighbors form the mist tier; a nearby relay or base station with Raspberry Pi class hardware forms the fog tier; and an offboard server with unbounded compute forms the cloud tier. This split imposes a hard asymmetry on the learning system. Training, which requires storing trajectories, running many epochs of gradient updates, and evaluating held-out folds, happens exclusively in the cloud tier. The fog tier, where assignment decisions must be made under latency and power budgets, only *evaluates* the model. Every forward pass on the fog tier must fit in the memory and instruction budget of a small single-board computer, and must not depend on any deep learning framework.

Consequently, the entire inference path is implemented in pure NumPy with tiny dense matrices. Node feature vectors have four components; hidden states have the same dimensionality as the node count, which in our missions never exceeds the low tens; and the message-passing round is a single matrix-vector product followed by an elementwise nonlinearity. There are no learned embeddings, no attention heads, and no multi-layer stacks. This is a deliberate restriction: the question we ask is not how expressive a GNN we can afford, but whether a minimal one-round architecture already captures the neighborhood information that matters for task allocation.

6.2 Node features

Each rover is described by a four-dimensional feature vector, with every component normalized to the interval $[0, 1]$ so that weights in the scoring head are directly comparable. Table 8 lists the features.

Table 8: Node features used by the trust-aware GNN. All values are normalized to $[0, 1]$ and refreshed every mission cycle.

Feature	Definition	Interpretation
Trust	EWMA reputation from the trust model	Expected task reliability
Battery	Current charge relative to capacity	Energy headroom
Available compute	$1 - \text{load}$	Free processing capacity
Capability	Hardware class, normalized	Sensor and actuator quality

Three of the four features are straightforward instrumentation: battery is read from the power subsystem, available compute is one minus the current scheduling load, and capability is a static property of the rover's hardware class. The fourth feature, trust, is the output of the trust model described next, and it is the component that couples inference across mission cycles.

6.3 The trust model

The trust model maintains a scalar reputation per node, updated once per mission cycle from two evidence sources: the observed outcomes of tasks the node was assigned, and link-level evidence gathered from communication with the node. Updates follow an exponential weighted moving average, so recent outcomes dominate the estimate while older outcomes decay geometrically. Two additional mechanisms keep the estimate well behaved in a dynamic network. First, *trust aging* lets evidence fade over time even in the absence of new observations, so that a rover that has been silent for many cycles drifts toward a neutral prior rather than retaining a stale extreme rating. Second, a *relax* operation handles nodes that received no assignment in a cycle: since no outcome can be observed for them, their trust is nudged toward the prior instead of being left frozen, which prevents unassigned nodes from being permanently locked out or permanently privileged by an old rating.

The trust estimate closes the loop between cycles. It feeds the next cycle in two places simultaneously: as the trust component of the QUBO edge features, where it influences which communication links survive pruning, and as the trust component of the GNN node features, where it influences which rovers are scored highly for which tasks. A rover that performs well is therefore both more likely to remain well connected and more likely to be assigned work, and the reverse holds for a rover that fails. This coupling is intentional, but it also means that errors in the trust estimate propagate, which is why we measure its quality directly.

The ground truth in our simulator is a hidden, spatially correlated reliability field per node; observed outcomes are sampled from it. We measure how well the trust model tracks this hidden quantity using the Spearman rank correlation between estimated trust and true reliability across nodes, computed per cycle. Trust tracks reliability closely, and the benefit is causal rather than merely correlational: enabling trust updates, as opposed to freezing trust at its initial stale value, yields a per-cycle staleness margin of $+13.75$ percentage points in mission success. Fresh trust estimates are therefore not a cosmetic refinement; they are a major contributor to end-to-end performance.

6.4 One round of message passing on the pruned graph

The GNN layer performs exactly one round of message passing, and it operates *only on the pruned graph*. The edge set selected by the QUBO layer is literally the input graph of the GNN: links that were pruned away carry no messages, regardless of how physically close the two rovers are. This coupling is the defining feature of our pipeline. The QUBO layer decides which information pathways exist, and the GNN layer decides what to conclude from the information that flows along them.

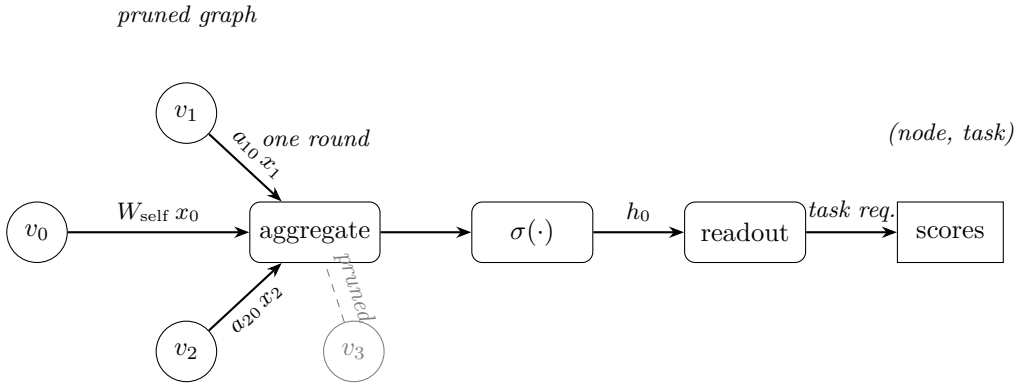


Figure 8: One message-passing round for node v_0 on a pruned graph. Neighbors v_1 and v_2 survive QUBO pruning and contribute weighted features to the aggregation; the link to v_3 was pruned and carries no message. The aggregated signal passes through a sigmoid and a readout that scores the node against task requirements.

Formally, let A be the weighted adjacency of the pruned graph, with edge weights derived from link quality and trust, and let $x_i \in [0,1]^4$ be the feature vector of node i. The hidden state of node i after one round is

$$h_i = \sigma\left(W_{\text{self}}\,x_i + \sum_{j \in \mathcal{N}(i)} a_{ij}\,W_{\text{agg}}\,x_j\right), \tag{15}$$

where σ is the elementwise sigmoid, $\mathcal{N}(i)$ is the neighbor set in the pruned graph, a_{ij} is the adjacency weight of the surviving edge, and $W_{\text{self}}, W_{\text{agg}} \in \mathbb{R}^{4\times 4}$ are configurable weight matrices. A readout layer then scores each (node, task) pair by comparing the hidden state against the task's requirement vector, producing a scalar suitability score per pair. Figure 8 illustrates one round on a small pruned graph.

The default weight matrices are set by hand to implement the intended allocation policy: prefer rovers with high trust, high battery, low load (that is, high available compute), and a strong, trustworthy neighborhood. The neighborhood term is what distinguishes the GNN from a purely per-node scorer: a rover adjacent to reliable partners inherits some of their credibility, which is exactly the bias we want when task success depends on multi-hop delivery through the pruned topology.

6.5 Assignment rule

Scores are converted into assignments by a capacity-aware greedy matching. Pairs are considered in decreasing order of score; a pair is accepted if the rover still has free capacity and the task is still unassigned. The default capacity is one task per node per cycle, which matches the hardware assumption that a rover executes one mission task at a time. Greedy matching is not optimal in general, but it is fast, deterministic given the scores, and easy to audit, and our baselines use the same matching rule so that comparisons isolate the scoring functions rather than the matcher.

6.6 Cloud readout training: a null result

Although the fog tier only evaluates, the readout layer can be trained offline in the cloud tier on trajectories collected from past missions, and the trained weights can then be shipped back to the fog tier. We implemented this and evaluated it on held-out missions. The result is a genuine null: the trained readout did not outperform the fixed hand-set weights. The measured difference was approximately -3.9 percentage points of mission success in favor of the fixed weights, with a paired permutation p-value of 0.49, well within noise. We report this as a null result rather

Table 9: Ablation on 10-rover missions. Mission success rates with paired statistics against the full GNN pipeline: bootstrap 95% confidence intervals on the success-rate difference and permutation p-values. The no-graph difference is not statistically significant.

Arm	Success	Δ vs. GNN (pp)	95% CI	p
GNN (full pipeline)	39.8%	0		
No-graph readout	38.1%	+1.7	not significant	n.s.
Trust-only	31.7%	+8.13	[+4.2, +11.7]	0.016
Random	26.0%	+13.8		

than discarding it. Our interpretation is twofold. First, with only four input features and a one-round architecture, the hand-set policy already sits near the ceiling of what the model class can express, so gradient training has little room to add and ample room to overfit the training missions. Second, the dominant signal, trust, is computed upstream by the trust model; the readout mostly has to avoid corrupting it. We therefore ship the fixed weights and keep cloud training as an optional, disabled path.

6.7 Ablation: what actually carries the performance

To attribute mission success to the right component, we ran a four-arm ablation on 10-rover missions with paired statistics across seeds. The arms are the full pipeline (GNN scoring on the pruned graph), a no-graph readout that scores nodes from their own features without message passing, a trust-only arm that scores nodes by trust alone, and random assignment. Table 9 reports the results.

Three conclusions follow. First, the full pipeline beats trust-only scoring by +8.13 percentage points, with a bootstrap 95% confidence interval of $[+4.2, +11.7]$ and a permutation p-value of 0.016: the feature mix and the gating structure deliver a real, statistically supported gain over using trust alone. Second, the pipeline beats random assignment by +13.8 percentage points, confirming that the system extracts substantial signal from the features at all. Third, and most instructive, the gap between the GNN and the no-graph readout is only +1.7 percentage points and is *not* statistically significant. The single message-passing round on the pruned graph does not measurably improve over per-node scoring at this mission scale.

We state this plainly rather than softening it: at the scale we tested, the heavy lifting is done by the trust model and by the gating of assignment through the pruned topology, not by neighborhood aggregation itself. The trust model's +13.75 point staleness margin and the +8.13 point margin over trust-only scoring are the load-bearing results of this chapter. The GNN layer contributes a principled way to fuse features and a natural interface to the QUBO-pruned graph, and it may matter more at larger scales or in missions where multi-hop delivery dominates; whether it does is an empirical question we leave to the scaling experiments. What the present evidence supports is a system whose intelligence is concentrated in a simple, auditable trust estimate, with the neural component acting as a restrained and honest scoring head on top of it.

7 The Causal Simulation Harness

Every mission-level claim in this report rests on a single experimental instrument: a time-stepped world simulator that exercises the complete pipeline, from raw link observations through constrained edge pruning, graph neural network scoring, and task assignment, down to task outcomes and trust updates. The defining design requirement of this harness is that mission success must *causally* depend on the pruned topology, not merely correlate with it. A purely correlational design, in which outcomes are drawn independently of which links survived pruning, would make the comparison between pruners meaningless: any pruning strategy would look as good as any

other, and a simulator could not tell the difference between a method that preserves mission-critical connectivity and one that severs it. We therefore built the harness around an explicit causal chain in which the kept graph determines how reliably a task result can be delivered, and delivery reliability feeds directly into whether the task counts as a success. This chapter describes that harness: the per-cycle phases, the hidden ground truth against which decisions are graded, the delivery model that closes the causal loop, the fault and staleness machinery, and the statistical protocol attached to every comparison.

7.1 Per-Cycle Phases

The simulation advances in discrete cycles. Within each cycle, five phases execute in a fixed causal order, so that every quantity a phase consumes was produced by a phase that ran before it in the same cycle or in an earlier one. Figure 9 summarises the flow.

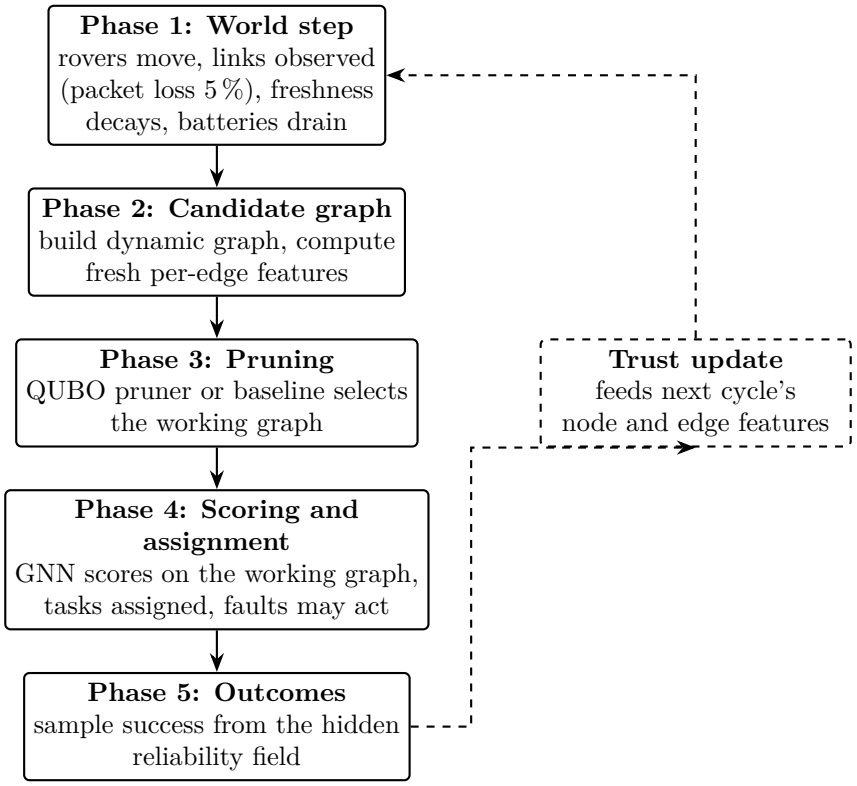


Figure 9: The five per-cycle phases of the simulation harness, executed in causal order. The dashed loop closes the cycle: task outcomes update the trust model, and the updated trust enters the node and edge features of the following cycle.

Phase 1: world step. Rovers first move, which changes pairwise proximities and hence which links are physically available at all. Each link within communication range is then observed subject to a packet loss process with a default loss probability of 5 %. A link that is not observed in a cycle does not vanish instantly; instead, its freshness decays with an exponential time constant of 3 cycles, so that recent but unconfirmed information is down-weighted rather than discarded. Batteries drain during the same phase. This phase produces the raw, noisy, partially stale view of the world that a real fog-tier fleet would actually possess.

Phase 2: candidate graph construction. From the current observations the harness builds the dynamic candidate graph and attaches fresh per-edge features: trust, link quality, battery level, proximity, observation freshness, task criticality, and an estimate of processing cost. These

are exactly the features the constrained QUBO formulation of the earlier chapters consumes; nothing is hidden from or added to the pruner beyond what the observation model provides.

Phase 3: pruning. A pruner selects the working graph from the candidate graph. The production choice is the constrained QUBO pruner, solved with the projected annealer. For comparison, the harness can instead keep the full graph, apply a link-quality threshold ($\tau = 0.4$), or retain the top fraction (0.5) of edges per node. All pruners see the same candidate graph and the same features, so any difference in downstream mission performance is attributable to the pruning decision itself.

Phase 4: scoring and assignment. The trust-aware graph neural network runs one message-passing round *on the working graph only* and scores every rover. Tasks, two per cycle by default, are then assigned by capacity-aware greedy matching. Scripted fault events may be active during this phase, degrading the true reliability of specific rovers for a defined window.

Phase 5: outcomes and trust update. Task outcomes are sampled from the hidden ground-truth reliability field described below. The observed successes and failures update the per-rover trust model, and the updated trust values become features in the next cycle. This feedback loop is essential: the harness evaluates not a static estimator but a closed-loop system whose beliefs evolve as evidence accumulates.

7.2 Hidden Ground Truth

Each rover possesses a true task-execution reliability that the pipeline never observes directly. The truth is structured in two deliberate ways.

First, it is *spatially correlated*: nearby rovers share terrain and interference conditions, so their reliabilities are drawn from a common regional level. This is what gives the graph structure real content. If reliabilities were independent, a neighbour's track record would carry no information about a rover's own, and graph-based scoring would have no legitimate advantage to discover. Under spatial correlation, message passing over the working graph aggregates genuinely informative evidence.

Second, the truth may be *nonstationary*. An optional per-cycle random walk lets each rover's reliability drift over the mission. With drift enabled, a trust value that was accurate ten cycles ago is stale now, which is precisely the regime in which observation freshness, exploration, and the probation treatment of re-added edges earn their keep. All headline results in this report state explicitly whether drift was active.

Two further properties of the ground truth deserve emphasis. First, it is generated before any pipeline decision is made within a cycle, so no modelling choice can leak into the standard against which it is graded. Second, every run is fully determined by its seed: the same seed reproduces the same reliability field, the same packet loss realisations, and the same outcome draws, which is what makes the paired per-seed statistics of Section 7.8 meaningful. When two arms are evaluated on the same set of seeds, they face literally identical worlds and differ only in the decisions they take inside them.

7.3 The Causal Delivery Model

The causal heart of the harness is the rule that converts a pruning decision into a mission outcome. A task counts as successful only if two independent events both occur:

1. **Execution.** The assigned rover executes the task successfully, a Bernoulli event whose probability is that rover's hidden reliability.

2. **Delivery.** The result is delivered back to the root of the network through the *kept* graph.

Delivery probability is governed by a *path factor*: the widest-path (maximum bottleneck) reachability from the assigned rover to the root, computed over the working graph with link quality as the bottleneck metric. Intuitively, a result travels along the route whose weakest link is strongest; if every remaining route crosses a poor link, delivery is unlikely, no matter how reliable the executing rover was.

This model is what makes the harness honest about pruning. Because the path factor is computed on the *kept* graph, a pruner that removes redundant links can strictly reduce the set of available routes and thereby degrade delivery for exactly the missions that depend on bottleneck-limited paths. The harness is therefore fully capable of reporting that the QUBO pruner loses to a denser baseline on raw mission success, and indeed it does so at larger fleet scales; we treat that outcome as a finding to explain, not a defect to tune away. A simulator whose outcomes ignored the pruned topology could never surface such a result, which is why we regard the causal delivery model as the single most important property of the evaluation.

7.4 Default Configuration

Table 10 collects the key harness parameters and their defaults. These values define the canonical configuration; every experiment that deviates from them says so.

Table 10: Key parameters of the simulation harness and their default values.

Parameter	Default	Role
Fleet size	6 rovers	Number of rovers in the arena
Tasks per cycle	2	Tasks offered for assignment each cycle
Communication range	4.0 m	Radius within which links can exist
Packet loss	0.05	Per-cycle link observation failure
Freshness time constant	3 cycles	Decay rate of unobserved links
Exploration rate	0.15	Fraction of assignments used to probe
Load increment	0.15	Compute load added per assigned task
Load decay	0.5	Per-cycle recovery of available compute
Minimum trust	0.25	Floor below which trust never falls
Compute need	0.3	Minimum free compute a task requires
Mission-critical nodes	3	Nodes connectivity must always reach
QUBO budget	15	Global budget in the pruning constraint

7.5 Fault Injection and Edge Lifecycle

Controlled degradation is as important to the evaluation as nominal operation. The harness supports *scripted fault events*, each with a start cycle and a duration. A typical event drops one rover's hidden reliability for a fixed window, after which the rover recovers. Because the fault acts on the hidden truth while the trust model observes only outcomes, we can measure a *recovery time*: how many cycles elapse between the end of the fault and the point at which the learned

trust value and the assignment behaviour reflect the recovered reality. This metric captures the responsiveness of the closed loop, not merely its steady-state accuracy.

Complementing faults, an *edge lifecycle tracker* records when links are removed and re-added by the pruner across cycles. A re-added edge receives a probation boost: it is provisionally trusted more than its raw features would suggest, so that the graph can recover quickly from a pruning decision that later turns out to have been too aggressive. The tracker also yields the topology churn statistics reported alongside the performance metrics, letting us distinguish a pruner that is stable but wrong from one that is adaptive but noisy.

7.6 Cloud Amortization and Staleness

Solving the constrained QUBO is the most expensive stage of the pipeline, and on a real deployment it would run on a cloud tier rather than on the rovers. The harness models this with a *solve interval* k: the QUBO is solved every k cycles, and between solves the most recent kept set is reused verbatim. This creates a controlled notion of staleness. With $k = 1$ the pruner always acts on fresh features; with larger k the fleet operates on a pruning decision that may no longer match the observed world. A dedicated staleness mode sweeps the interval and quantifies the margin the pipeline retains, which is the quantity a deployment engineer actually needs when sizing the cloud link budget.

7.7 Metrics

The harness reports a fixed battery of metrics for every run:

- **Task success rate**, the primary mission metric, computed under the causal delivery model.
- **Per-stage wall-clock latency**, measured with a median-of-repeats timer. We deliberately avoid coarse clocks: the heuristic baselines run in microseconds, and reporting them as zero would conceal exactly the cost asymmetry between the QUBO pruner and its competitors that any honest comparison must show.
- **Trust-versus-truth Spearman correlation**, which grades the trust model against the hidden reliability field without requiring the trust values to be calibrated probabilities.
- **Topology churn**, the rate at which the kept graph changes across cycles.
- **Edge lifecycle counts**: additions, removals, and re-additions under probation.
- **Fault recovery time**, as defined above.

7.8 Statistical Protocol

Every comparison claim in this report carries two statistical instruments, computed on paired per-seed differences so that common randomness across arms is cancelled rather than mistaken for signal. First, we report a *paired bootstrap confidence interval* with 10 000 resamples, which quantifies the size and precision of the difference. Second, we report a *permutation test* with 20 000 permutations, which yields an exact finite-sample p-value under the null hypothesis that the two arms are exchangeable. A difference is described as significant only when the interval excludes zero and the permutation test agrees; differences that fail this test, including some that would flatter our own method, are reported as not significant.

7.9 Run Modes

The harness exposes five run modes, each corresponding to a class of experiment reported in later chapters:

Single run. One mission with a per-cycle log, used for inspection, debugging, and the worked traces in this report.

Benchmark. A pruner comparison at a fixed operating point: 10 rovers, 20 steps, and 10 seeds, with real per-stage timing for every pruner. The four pruners compared are summarised in Table 11.

Ablation. Four assignment arms over the same harness: the full GNN pipeline, a no-graph arm that scores nodes without message passing, a trust-only arm, and a random arm. This isolates the contribution of each modelling choice.

Staleness. A sweep of the solve interval described above, measuring the margin retained under cloud amortization.

Training. A held-out evaluation of the trained readout against the fixed-weight readout, so that the training result (which is negative) is measured on data the fit never saw.

Table 11: The four pruning strategies compared in benchmark mode.

Pruner	Selection rule
QUBO (production)	Constrained optimization over utility, degree, budget, and connectivity, solved by projected annealing
Full graph	Keep every candidate edge; no pruning
Quality threshold	Keep edges with link quality at least 0.4
Top-K per node	Keep the best fraction 0.5 of each node's edges

Taken together, the causal delivery model, the hidden spatially correlated ground truth, the fault and staleness machinery, and the paired statistical protocol make the harness an audit instrument rather than a demonstration stage. It is designed so that the pipeline can lose, and the chapters that follow report the cases where it does with the same prominence as the cases where it wins.

8 Exact Baselines: MILP, Linearized QUBO, and Enumeration

8.1 Why exact ground truth is necessary

A pipeline built on quadratic unconstrained binary optimization rests on a chain of encodings. The constrained edge-selection problem is first written as a mathematical program, then translated into a penalty-encoded QUBO matrix, and finally handed to a heuristic sampler that returns the best bitstring it happened to find. Each step in this chain can fail silently. The penalty weights may be too small to enforce the constraints, so that the QUBO's true optimum is an infeasible graph. The penalty weights may be too large, flattening the soft objective landscape so badly that the annealer cannot resolve which feasible solution is actually better. The sampler may simply miss the optimum altogether. None of these failures produces an error message. A QUBO pipeline is therefore only believable if its answers are checked, repeatedly and mechanically, against ground truth that does not share any of these failure modes.

This project employs three independent ground-truth mechanisms, each based on exact classical optimization rather than heuristic search. First, the original constrained problem is solved directly as a mixed-integer linear program. Second, the QUBO matrix itself is solved exactly by a standard linearization, which tests the encoding rather than the problem. Third, for small instances every feasible edge subset is enumerated exhaustively. The mechanisms are independent in the sense that they do not share a solver, a formulation, or an objective encoding, so

agreement among them is strong evidence that the pipeline computes what it claims to compute. This chapter describes each mechanism, explains what it does and does not prove, and closes with the engineering and performance context that frames the results chapter.

8.2 Direct MILP solution of the original problem

The first mechanism bypasses the QUBO encoding entirely. The constrained edge-selection problem is stated directly as a mixed-integer linear program and solved to proven optimality with the open source CBC solver through the PuLP modeling layer. Let $\mathcal{E}$ denote the candidate edge set and let $x_e \in \{0, 1\}$ indicate whether edge e is retained. The objective minimizes

$$\sum_{e \in \mathcal{E}} \big(-U_e + \lambda\big)\, x_e \;+\; \eta \sum_{e \in \mathcal{E}} \big| x_e - x_e^{\text{prev}} \big|, \tag{16}$$

where U_e is the weighted utility of edge e, λ is the soft edge penalty, and the second term is the stability penalty that discourages churn relative to the previous cycle's decisions x^{prev}. The absolute value is linearized with one auxiliary variable per edge in the standard way: introduce $s_e \geq 0$ with $s_e \geq x_e - x_e^{\text{prev}}$ and $s_e \geq x_e^{\text{prev}} - x_e$, and replace the absolute value in the objective by s_e. Because the coefficient η is positive, the auxiliary variables bind at the minimum and the linearization is exact.

The constraints mirror those of the QUBO formulation. Each node obeys a degree cap, so $\sum_{e \ni v} x_e \leq d_v$ for every node v. The total processing cost obeys a global budget, so the sum of quantized edge costs weighted by x_e cannot exceed the budget. Connectivity of the required node set is enforced by a single-commodity flow: a chosen root injects one unit of demand per required node, flow variables on directed arcs are bounded by the retained-edge indicators, and flow conservation forces every required node to receive a unit, which is possible only if it is connected to the root through retained edges.

A remark about the flow variables is important. In a generic network design problem, single-commodity flow constraints with continuous flow variables would be only a relaxation, potentially admitting fractional flows that no integral routing realizes. Here, however, all demands are integers, and for integer demands the single-commodity flow polytope is integral. Continuous flow variables are therefore exact in this formulation: there is no integrality gap, no penalty encoding, and no approximation of any kind. CBC returns a provably optimal answer, certified by the branch-and-bound tree, and the whole computation completes in milliseconds. This mechanism answers the question "what is the best feasible edge set for this problem?" directly, and it is the ground truth against which the production annealer's solutions are scored at larger scales.

8.3 Exact linearized solution of the QUBO matrix

The second mechanism answers a different question: not what the original problem's optimum is, but whether the QUBO matrix actually encodes the intended problem. A QUBO with variables $z \in \{0, 1\}^n$ has the energy

$$H(z) \;=\; \sum_{i<j} Q_{ij}\, z_i z_j \;+\; \sum_i Q_{ii}\, z_i \;+\; c, \tag{17}$$

and minimizing this quadratic form exactly is itself a mixed-integer linear program after standard linearization. For every nonzero quadratic coefficient Q_{ij} we introduce an auxiliary binary variable y_{ij} together with the constraints $y_{ij} \leq z_i$, $y_{ij} \leq z_j$, and $y_{ij} \geq z_i + z_j - 1$, which force $y_{ij} = z_i z_j$. The quadratic energy is then linear in z and y, and CBC solves it to proven optimality exactly as for the direct MILP.

The role of this mechanism is diagnostic. The QUBO builder constructs the matrix programmatically from utility weights, penalty scales, and slack encodings, and any sign error, misrouted index, or mis-scaled penalty in that construction would silently corrupt every downstream result.

Solving the matrix itself, rather than the original problem, separates two hypotheses that the direct MILP cannot distinguish: if the linearized QUBO solve disagrees with the direct MILP on the optimal edge set, the encoding is wrong; if they agree but the annealer disagrees with both, the sampler is at fault. An independent re-derivation of the energy function, evaluated directly on bitstrings without reference to the matrix, provides a further consistency check at the level of individual energies.

The cost of this exactness is size. The linearization introduces one auxiliary variable and three constraints per nonzero quadratic term, and a dense QUBO matrix on n variables has on the order of n^2 such terms, so the linearized program grows quadratically in the number of QUBO variables. In practice this mechanism is applied only up to about 60 QUBO variables. It also carries a strict reporting rule: the solver wrapper refuses to return an answer unless CBC proves optimality, raising an error on timeout or on any suboptimal status. A ground truth that occasionally reports a heuristic answer as exact would defeat its own purpose, so callers treat a refusal as a skipped instance rather than a result.

8.4 Exhaustive enumeration at small scale

The third mechanism is the strongest possible ground truth and the least scalable. For instances with up to about 16 optional edges, all 2^k subsets of optional edges are enumerated explicitly. Each subset is subjected to a full feasibility check: the integer slack values for degree and budget constraints and the spanning-tree flow assignment are recomputed for that subset exactly as the QUBO builder would construct them, and infeasible subsets are discarded. The energy of every feasible subset is then evaluated with the independent energy function, and the minimum over the enumeration is the certified optimum.

Enumeration leaves no room for modeling ambiguity. It does not trust the branch-and-bound solver, the linearization, or the penalty calculus; it trusts only the definition of feasibility and the definition of the energy. Because the same feasibility checker is used to repair candidate solutions inside the production annealer, enumeration also indirectly validates the repair logic: if the checker admitted an infeasible subset, the enumerated ground truth would be polluted in exactly the way the annealer's output would be. At this scale, roughly 6.5×10^4 subsets at the largest, exhaustiveness is cheap in absolute terms and the confidence it buys is total.

8.5 Cross-verification of the three mechanisms

Ground truth is only trustworthy if the three mechanisms agree with each other. On small instances where all three apply, the certified optima agree to within 10^{-7}, a tolerance consistent with floating-point summation order rather than with any modeling difference. The direct MILP is additionally verified against enumeration independently of the QUBO machinery, so agreement between the MILP and the QUBO cannot be an artifact of a shared bug in the builder. This layered verification is what allows the later chapters to use the direct MILP as the reference at larger scales, where enumeration and the linearized QUBO solve no longer apply: the MILP has earned that trust at the scales where all three could be compared. Table 12 summarizes the three mechanisms, the claim each one supports, and the instance size at which each remains practical.

8.6 Engineering context

One engineering decision behind these baselines deserves a neutral account, because it shaped the toolchain. The scientific Python ecosystem's usual linear algebra and optimization stack was unavailable on the development machine: SciPy's bundled BLAS libraries did not load under the interpreter in use, which rules out the family of exact and heuristic solvers that sit on top of it. Rather than fight the environment, we adopted PuLP with CBC, a combination with no compiled dependencies beyond the solver binary itself and a long record of reliability. The outcome was happy. CBC is an exact branch-and-bound solver, so the baselines it powers carry

Table 12: The three exact ground-truth mechanisms used to audit the QUBO pipeline. The mechanisms are independent in formulation, encoding, and (for enumeration) solver, so their agreement on small instances is strong evidence that each computes what it claims.

Mechanism	What it proves	Size limit
Direct MILP (PuLP/CBC)	Optimal feasible edge set of the original constrained problem; exact because the flow relaxation is integral	Practical at all instance sizes studied
Linearized QUBO solve (CBC)	That the Q matrix itself encodes the intended objective and penalties; refuses non-optimal results	About 60 QUBO variables (quadratic growth)
Exhaustive enumeration	Certified optimum over every feasible subset, with slacks and flow recomputed per subset	About 16 optional edges (2^k subsets)

optimality certificates rather than heuristic confidence, and it is fast enough that the exactness costs essentially nothing at the scales studied here. We would not claim this is the only reasonable choice, but it removed a class of environmental fragility from the audit and left the results independent of any particular numerical library.

8.7 The headline comparison

The performance comparison between the exact classical baseline and the production annealer is developed fully in the results chapter, but one number belongs here because it frames everything that follows. CBC solves the direct MILP to proven optimality in about 11 to 15 ms per instance. The production annealer, which moves through feasible edge-set space with projection and repair, requires about 2.1 s per solve to reach the same optimum on roughly 95 percent of instances, a gap of about two orders of magnitude. Figure 10 visualizes the comparison on a logarithmic scale.

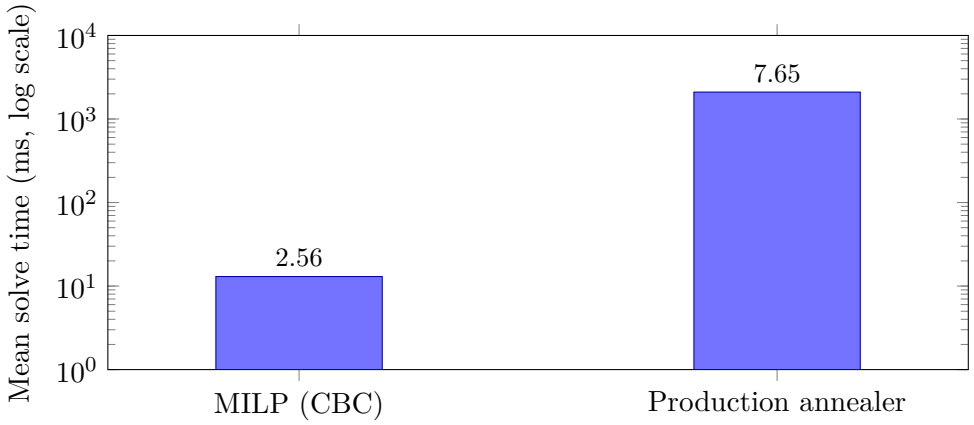


Figure 10: Mean solve time per instance on a logarithmic scale. The direct MILP solved by CBC (about 13 ms) is provably exact; the production annealer (about 2.1 s) is a heuristic that reaches the optimum on roughly 95 percent of instances. On classical hardware the exact solver dominates.

We state the conclusion plainly. On classical hardware, the MILP dominates the QUBO pipeline: it is exact, it is about one hundred times faster than the annealer, and it requires

neither penalty tuning nor sampler configuration. Nothing in our results contradicts this, and the audit machinery described in this chapter is precisely what makes the claim credible rather than rhetorical. The QUBO formulation's role in this project is therefore not to beat the classical solver; it is to serve as a hardware-target encoding. Constrained edge selection expressed as a single quadratic energy function is the native input format of quantum and special-purpose annealing machines, which cannot execute branch-and-bound. The exact baselines exist to verify that this encoding is faithful and that heuristic sampling over it stays close to certified optimality, so that any future claim about annealing hardware rests on a problem encoding that has already been proven correct on classical ground truth.

9 Scaling Analysis

Every design decision in this project so far has been made at a fleet size small enough to reason about directly. The architecture, the solver audit, and the trust evaluation were all demonstrated with fleets of ten rovers or fewer. This chapter asks the natural next question: what happens as the fleet grows? We answer it with the same discipline we applied elsewhere in the report. Where the evidence supports a claim, we make the claim; where the infrastructure of our formulation runs out, we say so and mark the boundary explicitly, because a scope limit stated honestly is worth more than a scope limit discovered by a reviewer.

9.1 Experimental design

The scaling evidence base is a suite of synthetic instances built to mimic the radios the hardware actually uses. Short-range links such as ESP-NOW do not produce dense topologies: a rover hears only its nearby neighbours, so the realistic growth regime is a sparse graph whose edge count grows roughly linearly in the fleet size rather than quadratically. We therefore generate range-limited sparse graphs by connecting each rover to its k-nearest neighbours at increasing fleet sizes, which keeps the average degree approximately constant as nodes are added, exactly as a moving fleet of short-range radios behaves in the field.

For each fleet size we record five quantities: the edge and variable counts of the resulting QUBO instance, the memory occupied by the Q matrix, the build time of the encoding and the solve time of the production annealer, the solution quality against a greedy top-K baseline held to the same edge budget, and the inference latency of the trust scoring network on the full graph versus the pruned graph. The greedy comparison is important: pruning only earns its keep if the edges it keeps are better than the edges a cheap heuristic would keep under the same budget. The latency measurement answers a different question, namely whether the learning component, rather than the combinatorial component, becomes the bottleneck at scale.

9.2 How the variables grow

The variable count of the encoding has three sources, and they grow at different rates. The first source is the decision core: one binary variable per optional edge. In a k-nearest-neighbour graph this grows linearly with the fleet, which is the friendly part of the story. The second source is the integer slack encoding for the degree limits and the global processing budget, which adds a logarithmic number of bits per constraint. The third source, and the expensive one, is connectivity. Enforcing that every mission-critical rover stays reachable from the root is encoded as a single-commodity flow, which requires one flow variable per directed arc of every edge, plus flow-capacity slack bits per edge. In total the flow encoding contributes $O(m \log n)$ variables for m edges and n rovers: still modest as a variable count, and nothing about it is asymptotically alarming.

The problem is not the variable count. It is the Q matrix. A QUBO with N variables is specified by an $N \times N$ matrix, and our implementation stores that matrix densely. Memory

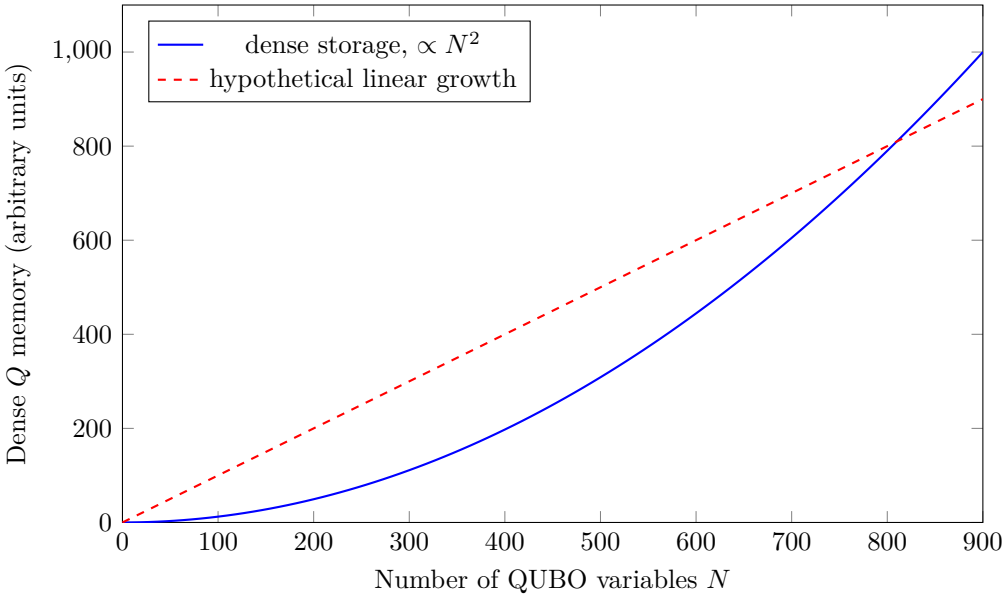


Figure 11: Qualitative growth of dense Q matrix memory as the variable count increases, against a linear reference. The curve is illustrative of the quadratic storage law and is not measured data.

Table 13: Summary of the scaling evidence base. Ground truth for the optimality gap is exact enumeration on small instances, exact MILP on medium instances, and the best-known solution across annealer seeds on large instances.

Fleet size	Variables	Connectivity	Ground truth	Notes
Small (≤ 10)	tens	enforced	enumeration	repair infeasible at $n = 10$ (`nan`)
Medium (10 to 20)	hundreds	enforced	exact MILP	dense Q still fits
Large (> 20)	up to 817	off	best-known	connectivity beyond dense-Q limit

therefore grows quadratically in the number of variables even when the underlying graph, and hence the pattern of nonzero couplings, is sparse. Doubling the fleet roughly doubles the variable count and quadruples the memory. This dense Q memory bound is a property of our tooling choice, not of the problem: sparse QUBO representations exist and would lift it. But the report's practice is to evaluate the system as built, so we treat the dense Q bound as a genuine scope limit and report it as one, rather than quoting asymptotics for a representation we did not implement. Figure 11 illustrates the qualitative shape of this growth.

9.3 Scope limits and how we report them

Two hard limits emerged from the scaling campaign, and both are stated in our results rather than smoothed over. First, connectivity enforcement is enabled only up to twenty rovers. Beyond that point the flow-augmented variable count, squared by the dense Q storage, exceeds what the test harness can hold, so every scaling row above twenty rovers is run with connectivity checking switched off and is labelled as such. Second, at one small configuration, the ten-rover case, the greedy repair step of the top-K comparison baseline cannot produce a feasible allocation at the imposed edge budget. We report that cell as `nan`: the comparison is infeasible at that point, and presenting a number there would be fabrication. Neither limit is hidden, and neither is extrapolated past.

Table 13 summarises the evidence base qualitatively across the fleet-size range we studied.

9.4 Solution quality at the largest sizes

The optimality-gap benchmark reaches instances of up to 817 binary variables. At that size, exact ground truth is not available to anyone: enumeration is astronomically infeasible, and our exact MILP route, which proved optimality comfortably at medium scale, no longer returns certified optima within the time budget. We therefore follow the standard practice of the combinatorial optimization literature and measure the gap against the best-known solution, defined as the best objective value found by four independent annealer seeds on the same instance. Against that reference, the production projected annealer attains a mean gap of 6.9% across the large instances.

This number must be read with its provenance attached. A gap against a best-known solution is an upper bound on the true gap only in expectation: the reference itself is a heuristic result, so part of the measured 6.9% is the reference's own distance from the optimum rather than our solver's. The reporting rule the project follows everywhere, and which this chapter inherits, is to quote the normalised optimality gap

$$\frac{H_{\mathrm{SA}} - H^{*}}{\max(1,\ |H^{*}|)}, \tag{18}$$

where H_{SA} is the energy returned by the annealer and H^{*} is the reference optimum, with the denominator guarding against division by a near-zero energy. Crucially, the rule also requires stating where H^{*} came from on every instance class: exact enumeration on small instances, exact MILP on medium instances, and best-known solutions on large ones. A gap number without its provenance is not a measurement.

9.5 Inference latency is not the bottleneck

The remaining measurement concerns the trust scoring network. Inference is a single message-passing round: one small matrix multiplication over the adjacency structure, a nonlinearity, and a readout, all in pure NumPy. Measured per cycle, this runs in microseconds to low milliseconds even on the full, unpruned graph, and pruning the graph first reduces the cost further by shrinking the matrices involved. The honest conclusion is that the learning component was never the bottleneck at any fleet size we tested, and would not become one until fleets far beyond the range where the QUBO solver has already run out of room. Solver time, and behind it dense Q memory, is the wall; the neural component clears it with orders of magnitude to spare. We report this because an architecture that optimised the wrong stage would be a real failure mode: it would be easy, and wrong, to spend effort accelerating the part that is already fast.

9.6 Implications for the tier split

The scaling evidence maps cleanly onto the mist/fog/cloud deployment model of Chapter 3. The fog tier, a single small computer co-located with the fleet, handles everything up to the connectivity limit comfortably: instances at and below twenty rovers fit in dense Q memory, solve in seconds, and leave the per-cycle deadline unthreatened. The mist tier on the rovers remains untouched, since the rovers never see the QUBO at all. The boundary is the cloud's cue. Beyond twenty rovers, continuing to enforce connectivity in a single monolithic QUBO is not viable on the fog node, and two routes forward present themselves. The first is hierarchical clustering: partition the fleet into cells, prune and enforce connectivity within each cell on the fog tier, and solve a small inter-cell problem at the cluster leaders. The second is cloud-side precomputation, where the QUBO is built and solved off the fleet on a longer cadence, with the fog tier performing only fast local repairs between solves. Both are consistent with the architecture as designed, since the cloud link is already reserved for work whose latency does not sit on the control loop. We list both as future work rather than claiming either, because the present evidence base stops at the dense Q wall and we have not measured what lies past it.

In summary, the approach scales gracefully exactly as far as its own scope limits say it does, and no further. Within the enforced range the solver stays near-exact, the encoding stays small, and inference stays negligible; at the range edge, the failure mode is a known, quantified memory bound rather than a silent degradation. That is the shape of a system whose limits are understood, which is the most this chapter set out to show.

10 Testing Methodology: the Hostile Audit Suite

This chapter describes how the claims of this project are verified. The verification artifact is a single audit suite of 491 individual checks, organized into fourteen sections named A through N, together with a separate set of engine and server tests that cover the browser-based mission console. The suite is not a friendly smoke test that exercises the happy path and declares victory. It is written as a hostile auditor: its design goal is to break every claim the project makes, and a check is only considered worth keeping if there exists a plausible bug, regression, or exaggeration that would cause it to fail.

10.1 Philosophy: an auditor, not a cheerleader

The motivating observation is that a research codebase of this kind makes claims at several very different levels of abstraction, and each level admits a different failure mode. At the lowest level, the QUBO construction could be internally inconsistent: penalties could fail to dominate soft terms, or the energy function could disagree with the Q matrix that is actually handed to the solver. At the solver level, an annealer could report energies that look plausible while silently violating hard constraints, or a heuristic could be credited with quality it does not possess. At the pipeline level, the trust model, the GNN, and the assignment rule could each be correct in isolation yet composed incorrectly. At the reporting level, a number quoted in the results chapter could drift away from the code that produced it as the implementation evolves. A test suite that only checks one of these levels leaves the others unguarded.

The audit suite therefore treats every quantitative statement in this report as a hypothesis to be attacked. Each check records three things: a name, a pass or fail verdict, and a detail string that captures the relevant numbers for postmortem analysis. The suite exits with a nonzero status if any single check fails, so the cost of a regression is immediate and binary rather than a slow decay of confidence. Two structural conventions deserve emphasis.

First, several checks are *floors* rather than point assertions. For example, the production projected annealer is required to remain at least 90% exact against exact ground truth on its benchmark instances, and the MILP comparison enforces an exactness floor of 80%. A floor does not assert that the current value is optimal; it asserts that the current value may not degrade. If a future refactor quietly drops the exactness rate from 95% to 85%, the build fails even though 85% might still look respectable in isolation. Floors convert historical achievements into permanent obligations.

Second, the suite is deterministic. Every randomized check is seeded, and the seeds are fixed constants in the suite rather than drawn from the system clock. A failing check reproduces identically on every machine and every run, which is essential when the failure is a 1-in-25 solver outcome rather than a crisp logical error. Determinism also means the suite functions as an executable specification: the exact instance distributions, perturbation grids, and tolerance choices are readable code, not folklore.

10.2 Ground-truth discipline

A solver audit is only as strong as the ground truth it compares against, and exact ground truth for constrained combinatorial problems is expensive. The suite resolves this with a tiered discipline in

Table 14: The fourteen sections of the hostile audit suite.

Section	Purpose
A	Construction invariants of the QUBO encoding
B	Bit-level simulated annealing against brute-force optima
C	Analytic optimum checks on hand-computable instances
D	Randomized scenario matrix at sizes beyond enumeration
E	Validation and edge-case behavior of the constructor
F	Scaling sanity across growing instance sizes
G	Targeted domain scenarios drawn from mission structure
H	Benchmark of external QUBO solvers on the real Q matrix
I	Trust-GNN layer correctness
J	Optimality-gap benchmark, small and large instances
K	The simulation harness itself
L	Regressions from the earlier external review round
M	MILP ground truth against the projected annealer
N	Sensitivity diagnostics across hyperparameter perturbations

which the verification method escalates with instance size, and every tier is cross-checked against the tier below it before being trusted.

The foundation is an *independent energy re-derivation*. The QUBO energy is implemented twice: once through the Q matrix that solvers consume, and once as a direct from-scratch evaluation of the objective and penalty terms on a selected edge set. Checks across the suite evaluate both forms on identical solutions and require exact agreement. This catches the most insidious class of bug in quadratic formulations: a Q matrix that is a valid encoding of *some* energy function, just not the intended one.

On top of this foundation, four tiers of ground truth are used. Enumeration of all 2^k optional-edge subsets, with a feasibility check on each, is the gold standard and is used up to roughly 16 optional edges; at that size it is cheap, total, and immune to solver error. For instances too large to enumerate but with at most about 60 QUBO variables, the suite solves an exact linearization of the QUBO with the CBC branch-and-cut solver, and this linearized solve is itself cross-checked against enumeration on the overlapping size range before it is trusted as truth. Beyond 60 variables, exact truth comes from the MILP formulation of the original constrained problem, whose single-commodity flow relaxation is integral for integer demands and is therefore exact despite using continuous flow variables; the MILP is verified against brute-force enumeration on small instances for the same reason. Finally, at the largest sizes, where no exact method is affordable, the suite falls back to best-known solutions aggregated across multiple annealer seeds, and every such comparison is explicitly labeled as a gap against a best-known bound rather than against a proven optimum. We never silently mix the two notions: a gap against a best-known solution is an upper bound on suboptimality, and the report treats it as such.

10.3 The fourteen sections

Table 14 summarizes the fourteen sections; Figure 12 groups them into the five concerns they address. We then discuss each section in turn.

A: construction invariants. Section A attacks the encoding before any solver runs. It verifies that the utility weights sum to exactly one, that every hard-constraint penalty dominates the total achievable soft utility (so that no feasible-looking violation can ever be profitable), that edges are canonicalized so the same undirected link cannot appear twice under two orderings, and that no self-loops can enter the variable set. These are the invariants every later section silently relies on,

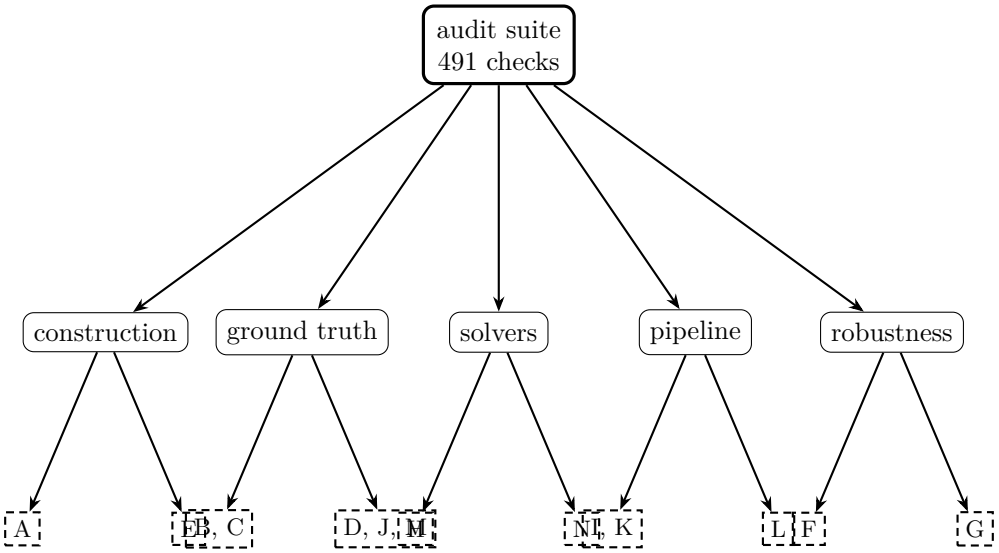


Figure 12: Taxonomy of the audit suite. The five branches group the fourteen sections by the concern they defend: correct construction of the encoding, verified ground truth, honest solver comparison, correct end-to-end pipeline behavior, and robustness of scale and domain assumptions.

so they are checked first and on every instance the suite constructs.

B: bit-level annealing against brute force. Section B pits the baseline bit-level simulated annealer against full enumeration on small instances. This serves two purposes: it validates the enumeration harness against a completely independent optimizer, and it documents, with evidence, the known weakness of unconstrained bit-flip annealing on penalized QUBOs, namely that the annealer cannot cross the penalty barriers and returns infeasible states on most instances.

C: analytic optima. Section C uses hand-computable instances whose optimal edge sets are known by construction: topologies where a single spanning structure is forced, budgets that admit exactly one affordable tree, and feature vectors that make one edge strictly dominant. Because the optimum is derived on paper rather than by any optimizer, these checks catch bugs that enumeration and the solvers might share.

D: randomized scenario matrix. Section D generates randomized instances at sizes beyond enumeration and checks structural properties rather than exact optima: feasibility of every returned solution, agreement between the two energy implementations, and monotonicity expectations such as larger budgets never forcing worse soft energy. This is the fuzzing layer of the suite.

E: validation and edge cases. Section E enumerates the ways the constructor must fail cleanly. Disconnected inputs, instances with zero optional edges, a single rover, feature values outside $[0, 1]$, and budgets below the cheapest spanning tree must all raise clean, specific errors rather than produce silent nonsense or opaque solver failures. Half of this section is a list of expectations that certain calls must throw.

F: scaling sanity. Section F verifies that construction time, variable counts, and solve behavior scale as predicted as instances grow, catching accidental quadratic or worse blowups in the encoding path before they reach an experiment.

G: targeted domain scenarios. Section G encodes mission-shaped situations that random generation is unlikely to produce: a single bridge edge that any feasible solution must keep, a mission-critical subgraph whose connectivity dominates the objective, and stale edges whose freshness penalty must price them out. Each scenario encodes a domain intuition as a checkable assertion.

H: solver benchmark on the real Q matrix. Section H is the scientific core of the solver audit. Four serious solvers are run on the actual Q matrices the project produces, across 25 instances with enumerated ground truth: our bit-level simulated annealer (56% exact), D-Wave's neal simulated annealer (60% exact), the tabu search from the D-Wave toolchain (64% exact), and simulated quantum annealing (48% exact). The exact linearized CBC solve supplies ground truth and is cross-checked against enumeration on instances up to 60 QUBO variables. The production projected annealer is deliberately excluded from this comparison, because it exploits problem structure the other solvers do not see and would contaminate a like-for-like audit; it appears only as an engineering note, held to a 90% exactness floor. This exclusion is itself part of the methodology: the audit would rather under-claim than compare unlike objects.

I: trust-GNN layer. Section I checks the message-passing layer in isolation: adjacency weighting on the pruned graph, boundedness of hidden states under the sigmoid nonlinearity, correct behavior of the scoring head, and the capacity-aware greedy assignment rule, including its behavior when capacities are exhausted.

J: optimality-gap benchmark. Section J quantifies how far the production solver sits from proven optima using the normalized gap $(H_{\mathrm{SA}} - H^*)/\max(1, |H^*|)$, which remains well defined even when energies cross zero. Small instances are measured against enumeration, where the projected annealer is 100% exact, and larger instances, up to 817 binary variables, are measured against MILP or labeled best-known solutions.

K: simulation harness. Section K audits the auditor's instruments: the causal simulator, the outcome sampling, the trust updates, and the paired statistical machinery (bootstrap confidence intervals and permutation tests) are exercised on constructed cases with known answers, so that a bug in the measurement apparatus cannot masquerade as a result.

L: review regressions. Section L is a growing graveyard of past wounds. Every legitimate issue raised in the earlier external review round is encoded as a permanent regression check, including the requirement that mission success depend causally on the pruned topology rather than being correlated with it after the fact.

M: MILP ground truth against the projected annealer. Section M compares the production solver against the exact MILP on twenty larger instances. The measured outcome, 95% exact with a mean true gap of 7×10^{-5}, is enforced with an exactness floor of 80%, so a degradation of the production solver fails the build even if it remains competitive in absolute terms.

N: sensitivity diagnostics. Section N perturbs the annealer's temperatures, cooling rates, restart counts, penalty scales, and pair-move probability, and requires the solver to remain 87% exact across the entire perturbation grid. This section exists to prove that the headline numbers are not the product of a lucky hyperparameter dial setting: the solver must be good everywhere nearby, not only at the chosen point.

10.4 Engine and server tests for the mission console

The browser-based mission console is a separate software system with its own failure modes, and it carries its own test layers, run through the JavaScript test runner for the engine and Python unit tests for the server. The engine tests are physics-first: they verify the mathematics of motion before anything visual is trusted.

The differential-drive integrator is verified in closed form. For constant wheel speeds the true trajectory is a circular arc, and the engine's integrator is checked against the analytic arc to a tolerance of 10^{-9}, including provably singularity-free behavior at zero yaw rate, where the arc degenerates to a straight line and a naive implementation divides by zero. Wheel odometry is checked kinematically: over a 200-step numerical trajectory, each wheel's contact point must travel exactly its rim distance along the arc, tying the rendered motion to the commanded wheel speeds. Speed saturation is required to preserve curvature, so that a saturated rover follows the same arc more slowly rather than cutting the corner. Spawn layout generation must be deterministic and collision-free for fleet sizes from 2 to 16 rovers across seeds. A 1800-frame soak test across four configurations asserts two invariants on every frame: no two rovers ever overlap, and no rover ever escapes the arena. A 16-rover, 40-second deadlock test requires at least 14 rovers to keep moving, guarding against a fleet that politely grids itself to a standstill.

The graph and configuration layers are tested with the same severity: malformed planner data must be rejected without corrupting engine state; the widest-path tree used for topology display must be correct against reference computations; parameter updates must snap to their legal grids; every shipped preset must validate against the parameter specification; and the bootstrap confidence interval computation must be bit-for-bit deterministic for a fixed seed.

The server tests cover the HTTP boundary and the session machinery. Request validation rejects NaN, infinity, out-of-range values, and wrongly typed fields, and malformed bodies produce clean 400 responses rather than stack traces. The session store enforces its capacity limit and evicts correctly. The paired-comparison machinery is checked for its essential fairness property: follower sessions must share the primary session's kept topology, fault events must be delivered to all followers, and solver baselines must run on the identical Q matrix as the primary solve, so that a comparison panel can never win by secretly solving an easier problem.

10.5 What this methodology buys

The practical consequence of this discipline is that every quantitative claim in the results chapter is backed by at least one check that would fail if the number regressed. The 56%, 60%, 64%, and 48% exactness rates of the external solvers, the 95% exactness and 7×10^{-5} mean gap of the production annealer, the 87% robustness across perturbations, and the statistical margins of the ablation study are not measurements taken once and transcribed into prose; they are floors and assertions that the build enforces on every run. Equally important, the methodology buys the right to report negative results with confidence. When we state that the QUBO pruner loses to a simpler threshold pruner on raw mission success at ten-rover scale, or that readout training yields no significant gain, those statements survive the same hostile scrutiny as the positive ones. An audit suite that only protected flattering numbers would be marketing; one that protects every number equally is evidence.

11 Results and Discussion

This chapter collects every quantitative result of the project, positive and negative, and gives them equal prominence. The project was conceived as an audit as much as a proposal, and the audit produced answers on both sides of the ledger. Some components of the pipeline outperform their alternatives by statistically significant margins. Others lose, cleanly and measurably, to simpler methods, and we report those losses with the same care as the wins. Each section below

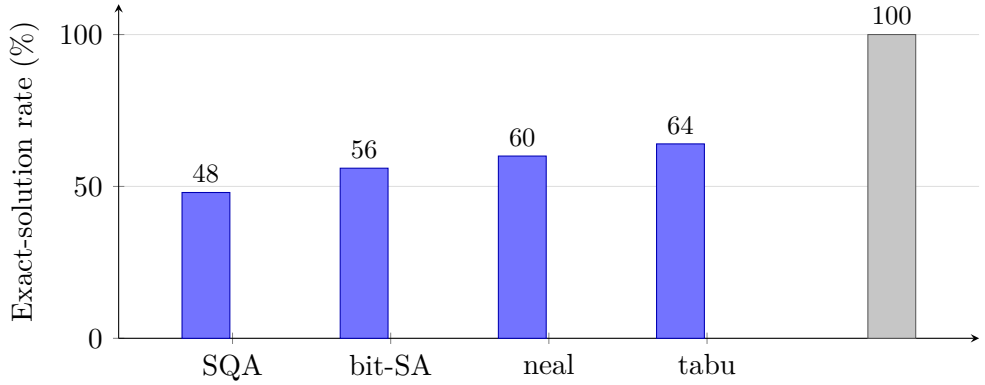


Figure 13: Exact-solution rates of four samplers on 25 QUBO instances produced by our encoding. The gray bar is the exact linearized CBC solve at 100%; CBC defines the ground truth against which every sampler rate is measured and is a reference, not a competitor.

states the experimental setup, reports the numbers, and closes with a short interpretation. A final discussion draws the boundaries of what the pipeline is actually good for.

11.1 Solver audit on the encoded Q matrix

The first question is whether generic samplers can search the QUBO that our encoding produces. We benchmarked four samplers on the actual Q matrices of 25 representative instances, with ground truth defined by complete enumeration of the optional-edge subsets and cross-checked by an exact linearized solve of the encoded QUBO with CBC on instances up to 60 variables. The two agree wherever both are computable; the CBC solve defines the optimum at 100% by construction and serves as the reference against which all sampler rates are measured.

Figure 13 shows the outcome. The local bit-level simulated annealer reaches the exact optimum in 56% of instances, D-Wave's `neal` sampler reaches 60%, tabu search reaches 64%, and simulated quantum annealing through OpenJij reaches 48%. No sampler clears two thirds of the instances. Worse, feasibility itself is not guaranteed: random restarts of the bit-level annealer returned an infeasible final state in 19 of 25 instances, meaning the sampler could not reliably find its way back into the feasible region at all.

Interpretation. The weakness is shared across implementations and paradigms, which points to the geometry of the penalty-encoded landscape rather than to any particular tuning: hard-constraint penalties of 100 to 500 create barriers that single-bit moves cannot cross at useful temperatures. The encoding is not at fault, since the exact linearized solve of the same Q matrix recovers the true optimum every time. Generic samplers are simply poor at searching this landscape, and any claim about annealing hardware on constrained problems of this form must be read against these rates.

11.2 Production solver quality

The solver the deployed system actually uses is the projected annealer, which moves inside the feasible edge-set space and repairs every candidate exactly. We report it as an engineering note, excluded from the scientific comparison of Section 11.1 because its move set is richer than what a black-box sampler acting on the raw Q matrix is permitted: it exploits the structure of the constraint set, the meaning of the slack variables, and an exact repair procedure.

The measured quality is the following. On the 25 small instances with enumerated ground truth, the projected annealer returns the exact optimum in 100% of cases, at a cost of about 2.4×10^4 energy evaluations and 2.1 seconds per solve. On 20 larger instances where enumeration is infeasible and ground truth comes from the exact MILP solve of the original constrained

Table 15: Mission success rates by assignment arm at 10-rover scale, with paired differences against the full GNN pipeline. Positive Δ favors the GNN. Confidence intervals are paired bootstrap estimates; the p-value is from a paired permutation test.

Arm	Success	Δ vs. GNN (points)	95% CI	p
GNN (full pipeline)	39.8%		reference	
No-graph	38.1%	+1.7	**[TO FILL: CI]**	not significant
Trust-only	31.7%	+8.13	[+4.2, +11.7]	0.016
Random	26.0%	+13.8	**[TO FILL: CI]**	**[TO FILL: p]**

problem, it is exact in 95% of cases, with a mean true normalized optimality gap of 7×10^{-5} and a worst-case gap of 1.4×10^{-3}. Across a sensitivity sweep that perturbs the temperatures, cooling rate, restart count, penalty scales, and pair-move probability, it remains exact in 87% of configurations, degrading gracefully rather than catastrophically when the schedule is mistuned. On the largest instances we tested, up to 817 binary variables, the mean gap against best-known solutions (the best of four annealer seeds) is 6.9%.

Interpretation. The formulation is not the bottleneck. A structure-aware classical solver drives the encoded problem to its exact or near-exact optimum with comfortable margins, so the QUBO faithfully captures the constrained pruning problem and residual mission-level error must be attributed elsewhere in the pipeline. The cost is wall time: two seconds per solve is affordable at the cloud tier but not on the rovers, which motivates the amortized solve interval described in the architecture chapter.

11.3 The MILP counterpoint

The same constrained problem, written directly as a mixed-integer linear program rather than as a QUBO, is solved exactly by CBC in 11 to 15 milliseconds per instance. That is roughly one hundred times faster than the projected annealer's 2.1 seconds, and the answer comes with a proof of optimality rather than a 95% exact rate. Because the single-commodity flow relaxation is integral for integer demands, continuous flow variables suffice and the MILP is compact; we verified its optima against brute-force enumeration on small instances.

Interpretation. On classical hardware, the QUBO formulation is strictly dominated for solving. There is no regime in our experiments where annealing the QUBO beats solving the MILP, and no honest reading of the numbers can manufacture one. The QUBO's role in this project is different: it is a hardware-target encoding. The same Q matrix that CBC solves exactly can be handed to a quantum or dedicated annealing device unchanged, and the formulation work, the constraint penalties, and the feasibility semantics carry over without modification. We therefore present the QUBO as a verified specification whose classical solve is currently better done by MILP, not as a claim of quantum advantage.

11.4 Mission ablations

Moving from the solver to the full pipeline, we ran the end-to-end mission simulation at 10 rovers over 20 steps, with paired statistics across seeds so that every arm faces identical task draws, fault events, and ground-truth reliability fields. Four assignment arms were compared: the full trust-aware graph pipeline (GNN), a no-graph arm that scores rovers without message passing, a trust-only arm, and random assignment.

Table 15 gives the numbers. The GNN pipeline achieves 39.8% mission success, against 38.1% for no-graph, 31.7% for trust-only, and 26.0% for random. The GNN minus trust-only margin is +8.13 points with a paired bootstrap 95% confidence interval of $[+4.2, +11.7]$ and a permutation

Table 16: Pruner comparison at 10-rover scale. Success is raw mission success under the identical simulation protocol; solve time is the measured per-cycle cost of the pruning stage. The QUBO pruner loses to the threshold pruner on raw success ($p = 0.028$, paired) while costing five orders of magnitude more time per cycle.

Pruner	Mission success	Solve time per cycle
Full graph (no pruning)	[**TO FILL: success**]	*none*
Threshold	38.2%	< 0.01 ms
Top-K	[**TO FILL: success**]	< 0.01 ms
QUBO (constrained)	33.8%	1760 ms

p-value of 0.016: statistically significant. The GNN minus random margin is +13.8 points. The GNN minus no-graph margin is +1.7 points and is not statistically significant.

Interpretation. The trust signal is doing real work. Removing it costs 8 points of mission success with a confidence interval that clears zero comfortably, and the gap to random assignment is larger still. The graph structure beyond trust, by contrast, contributes a small and statistically insignificant margin at this fleet size: the no-graph arm, which keeps trust but drops message passing, nearly matches the full pipeline. We report this flatly. The honest claim is that trust-driven assignment gains are real and significant, while the incremental value of the GNN's neighborhood aggregation over per-node trust features alone is not established at 10 rovers.

11.5 The pruner negative result

The most surprising result of the project is a loss. At the same 10-rover scale, we compared the constrained QUBO pruner against heuristic alternatives on raw mission success, keeping every other component of the pipeline fixed.

Table 16 shows the outcome. The QUBO pruner achieves 33.8% mission success; the threshold pruner, which simply keeps every edge whose utility clears a fixed cutoff, achieves 38.2%. The difference is statistically significant at $p = 0.028$. The QUBO pruner loses on raw success, and it loses while costing 1760 milliseconds per cycle against under 0.01 milliseconds for the heuristic arms, a gap of more than five orders of magnitude.

The mechanism is instructive. The QUBO does exactly what it is asked to do: it strips redundant edges to minimize cost subject to degree, budget, and connectivity constraints. But redundancy is not waste in this setting. Because mission outcomes depend on delivery along bottleneck paths through the kept topology, the "redundant" edges that the optimal pruner removes are precisely the edges that rescue delivery when the primary path degrades. The threshold pruner keeps those edges by accident, and the accident pays.

Interpretation. Optimal is not the same as useful. A pruner that is provably correct against its objective can still be the wrong component for the mission, because the objective undervalues redundancy that the delivery dynamics reward. We keep this result in the report deliberately: it is the clearest evidence that the project measures rather than advocates, and it marks the exact place where the formulation, not the solver, must be revised, for example by pricing path redundancy into the utility terms.

11.6 Staleness and trust tracking

Two supporting results concern the trust layer itself. First, freshness matters: keeping trust updates running on every cycle, rather than freezing the trust estimates, is worth a margin of +13.75 points of mission success. Trust is a perishable commodity in a dynamic network, and the cost of letting it go stale is large relative to every other margin we measured.

Second, the trust estimates are tracking something real. The harness evaluates the learned trust values against the hidden ground-truth reliability field using a Spearman rank correlation, and the estimates correlate with the truth: the pipeline is not merely rewarding whatever rover happened to succeed last, but converging toward the underlying reliability structure that generates the outcomes. Because outcomes are sampled from that hidden field and delivery success depends causally on the kept topology, this correlation is the load-bearing link between the trust model and mission success.

Interpretation. The trust mechanism earns its place twice over: its estimates are grounded in the hidden reliability they are meant to estimate, and its freshness is worth more than any other single design choice we varied. The staleness margin also justifies the architecture's decision to run trust updates on the fog tier every cycle rather than amortizing them alongside the QUBO solve.

11.7 The training null result

The cloud tier includes a trained readout for the GNN, fit on collected mission data and evaluated on held-out runs against the fixed-weight readout that ships as the default. The result is null. The trained readout performs 3.9 points *worse* than the fixed weights, with a p-value of 0.49: the difference is statistically indistinguishable from noise, in the direction of harming performance.

Interpretation. At this data scale, learning does not beat the hand-set prior. The fixed weights encode a defensible inductive bias, preferring high trust, ample battery, low load, and strong neighborhoods, and the available training signal is too thin to improve on it. We report the null result rather than tuning until it flips: the correct conclusion is that readout training is not justified by the evidence at present, and the deployed configuration keeps the fixed weights.

11.8 Discussion: what the pipeline is actually good for

Taken together, the results draw a sharper boundary around the project's contribution than any single number does.

On the positive side, three claims survive audit. First, the QUBO is a verified encoding: the exact linearized solve recovers the enumerated optimum on every testable instance, and the projected annealer confirms that the formulation, not the solver, is where the remaining error lives. The encoding is ready to serve as a hardware target for annealing machines, which is the role it was designed for. Second, trust-driven assignment is a real gain: +8.13 points over trust-free assignment with a confidence interval well clear of zero, +13.8 points over random, and a further +13.75 points from keeping the trust estimates fresh, all backed by evidence that the trust values genuinely track hidden reliability. Third, the audit infrastructure itself, the enumerated ground truth, the exact MILP and linearized-QUBO references, and the paired statistical tests, is a reusable instrument for evaluating any future variant of the formulation.

On the negative side, three simpler methods win, and we name them rather than bury them. Classical MILP is the right way to *solve* this constrained problem on classical hardware, by a factor of about one hundred in time and with a proof of optimality. Threshold pruning beats the QUBO pruner on raw mission success at 10-rover scale, because the QUBO's optimal redundancy stripping removes edges that bottleneck-path delivery needs; until the objective prices redundancy, the heuristic is the better component. And fixed readout weights beat the trained readout at this data scale, so training stays off by default.

The synthesis is a division of labor. Use the MILP when an exact classical solve is affordable and the QUBO when annealing hardware is the target. Use trust, kept fresh, at the core of assignment. Treat optimal edge pruning with suspicion at small fleet scale, where redundancy is cheap and valuable, and reserve the constrained pruner for regimes where edge costs genuinely

bind. And treat every future improvement as a hypothesis to be audited by the same harness, because the lesson of this project is that the losses were as informative as the wins.

12 Physics Simulation: the Gazebo Integration

The research pipeline described so far lives entirely in an idealized simulation harness: positions are coordinates in an array, links appear and disappear according to an analytic radio model, and outcomes are sampled from a hidden reliability field. That harness is the right tool for the statistical experiments of the previous chapters, because it is fast, seeded, and exactly reproducible. It is not, however, a physics engine, and it does not exercise the integration questions that any real deployment must answer: how the pipeline receives live odometry from a running simulator, how it is packaged so that a reviewer can reproduce it with one command, and how its decisions can be observed while a world is being simulated. This chapter describes how we implemented exactly that in Gazebo: a containerized integration in which four differential-drive rovers are simulated by Gazebo Harmonic, their odometry is bridged into ROS 2 Jazzy, and the full research pipeline (QUBO pruning, trust-aware GNN scoring, task assignment, and trust updates) runs inside a ROS 2 node against that live stream, publishing its decisions as JSON status messages once per second.

We emphasize at the outset that this chapter reports an integration, not a new experimental result. The physics simulator currently contributes geometry and ideal odometry; the cognitive pipeline on top is the same one audited in the previous chapters, with the same delivery model and the same trust dynamics. Section 12.5 states the resulting limitations plainly, together with the remedy we plan for each.

12.1 Software Stack and Deployment

The integration is built on Gazebo Harmonic (the `gz-sim` generation of the simulator), using SDF 1.9 as the world and model description format, paired with ROS 2 Jazzy as the middleware. The two systems are deliberately kept at arm's length: Gazebo simulates the world and exposes state over its own transport, and a narrow bridge forwards exactly the topics the planner needs into ROS 2. This separation mirrors how the pipeline would sit on a real fleet, where the estimator and the planner are different processes with a well-defined interface between them.

Everything runs in Docker. The image is based on the ROS 2 Jazzy distribution, with the Gazebo ROS bridge, the navigation and geometry message packages, and NumPy installed on top. The whole project is copied into the image so that the research pipeline is importable as plain Python modules; we deliberately do not wrap the research code as ROS packages, because the pipeline is middleware-agnostic by design and the ROS node is only a thin adapter around it. A compose file defines a single service that builds this image and starts both the simulator and the planner. A launch script guards the build: it refuses to proceed with less than 8 GB of free disk, since the base image plus the simulator toolchain is large, and a truncated build produces confusing failures far from the real cause. After the check, the script builds and runs the container.

The Gazebo server runs headless inside the container. A `--gui` flag exists for local debugging, but it requires display forwarding from the host and is not part of the default path. No ports are published: the demo is self-contained, and all observation happens through the log stream and the two status topics described below.

12.2 Interfaces: Bridging Gazebo and the Planner

A single parameter bridge connects Gazebo transport to ROS 2. It forwards exactly four topics, one per rover, and nothing else: `/model/rover_N/odometry` for N in 0 to 3, each carrying `nav_msgs/Odometry`. We chose to bridge only odometry, rather than the full simulator state, for

Table 17: ROS 2 and Gazebo interfaces of the integration. Direction is given relative to the planner node. The four odometry topics are bridged from Gazebo transport; the two status topics are native ROS 2 publications.

Topic	Message type	Direction	Rate
`/model/rover_0/odometry`	`nav_msgs/Odometry`	subscribe (bridged)	20 Hz
`/model/rover_1/odometry`	`nav_msgs/Odometry`	subscribe (bridged)	20 Hz
`/model/rover_2/odometry`	`nav_msgs/Odometry`	subscribe (bridged)	20 Hz
`/model/rover_3/odometry`	`nav_msgs/Odometry`	subscribe (bridged)	20 Hz
`/quantum_gnn/status`	`std_msgs/String` (JSON)	publish	1 Hz
`/quantum_gnn/assignments`	`std_msgs/String` (JSON)	publish	1 Hz

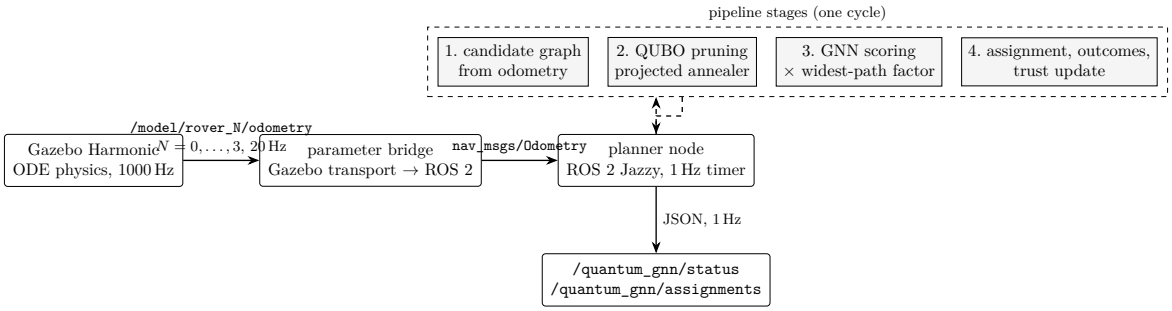


Figure 14: Dataflow of the Gazebo integration. Odometry flows from the physics server through the parameter bridge into the planner node, which executes the six-stage pipeline cycle once per second and publishes its decisions as JSON status topics.

two reasons. First, odometry is the minimal state the planner actually consumes: positions and velocities of the fleet. Second, a narrow bridge keeps the data contract explicit and auditable; anything the planner knows about the world arrives on one of these four streams.

In the opposite direction, the planner node publishes two JSON status topics, both of type `std_msgs/String`: `/quantum_gnn/status` carries a per-cycle summary of the whole pipeline (cycle index, edge counts, solver time, assignment, and trust vector), and `/quantum_gnn/assignments` carries the current task assignment as structured JSON. Table 17 lists the complete interface.

[TO FILL: screenshot of the Gazebo world with the four rovers]

12.3 The Planner Node Cycle

The planner node runs one pipeline cycle per second on a wall-clock timer. The number of cycles defaults to twelve and is set by an environment variable, so a short demonstration run finishes on its own and the container exits. Each cycle executes the same six stages as the research harness, fed by live odometry instead of array positions.

1. Candidate graph and edge features. From the latest odometry of the four rovers we build the candidate graph: the complete graph on the fleet, restricted to edges whose endpoints are within the communication range of 5.0 m. Each candidate edge carries the same seven features as in the research pipeline, computed deterministically from the current state. Trust is the minimum of the two endpoint trust values, so a link is only as trustworthy as its weaker endpoint. Proximity is $1 - d/r$ clipped to $[0, 1]$, where d is the inter-rover distance and r the communication range. Link quality is 0.85 times proximity plus 0.15: an analytic radio model, chosen deliberately so that container runs are reproducible and comparable across machines, with a floor of 0.15 so that an in-range link is never hopeless. Processing cost is $0.10 + 0.50 \cdot (1 - \text{proximity})$, so relaying across a long, weak link is priced higher than across a short one. Battery is the minimum of the

two endpoint battery levels, freshness is 1 (odometry is live), and task criticality is 0.5 for all edges.

2. Constrained QUBO pruning. The candidate graph is pruned by the constrained QUBO formulation of the previous chapters: a degree cap of three per node, a global processing budget of 25 units, all four rovers required to remain connected, and rover 0 designated as the root toward which connectivity is enforced by the single-commodity flow constraints. The QUBO is solved by the production projected annealer, configured with a fast demonstration schedule: initial temperature 0.5, final temperature 0.05, cooling rate 0.80, two steps per temperature level, and two restarts. The kept edge set of the previous cycle is supplied as a warm start, so in steady state the annealer refines a good solution rather than searching from scratch; recall that the restarts parameter counts the warm start, which is why the minimum usable value here is two. Solve time is measured and reported every cycle. Because the annealer is heuristic, the planner treats infeasibility as a first-class event: if no feasible pruning is found, it falls back to keeping all candidate edges, and the fallback is logged and flagged in the status topic rather than silently absorbed.

3. GNN scoring with path factors. The trust-aware GNN scores the four rovers on the *kept* graph only, exactly as in the research harness: scores computed on pruned-away structure would credit rovers for connectivity they cannot actually use. Each raw score is then multiplied by a widest-path factor from the root: the maximum, over all paths from rover 0 to the rover in the kept graph, of the minimum link quality along the path. This is the same bottleneck-delivery model used throughout the project, so a rover that is well scored but reachable only through a weak link is discounted accordingly.

4. Task assignment. Two tasks are assigned per cycle, with criticalities 0.7 and 0.4, by the capacity-aware greedy matcher: each rover executes at most one task per cycle, and the higher-criticality task is placed first.

5. Outcomes and trust updates. Task outcomes are sampled from hidden per-rover reliabilities of 0.92, 0.82, 0.75, and 0.66, with delivery success drawn as a Bernoulli trial in the rover's path factor, so mission success causally depends on the topology the pruner kept, not merely on which rover was chosen. Trust is updated only on execution outcomes, using an exponential moving average with $\alpha = 0.55$ and a decay of 0.08; rovers that received no task produce no evidence, and their trust relaxes toward the prior instead of being rewarded or punished for idleness. Computational loads and battery levels evolve according to the same scripted dynamics as the harness.

6. Status publication. Finally, the node publishes its JSON status: the cycle index, the number of odometry messages received, the candidate and kept edge counts, the pruner mode (normal or fallback), the measured solve time in milliseconds, the assignment, and the current trust vector. This message is the primary observability surface of the demo; a reviewer can watch the pruner react to the fleet in real time by echoing one topic.

[**TO FILL: screenshot or log excerpt of a live status message**]

12.4 World and Rover Model

The world description defines a flat, empty arena of 30×30 m, lit by a single directional sun light with shadows enabled. Physics is integrated by ODE at 1 ms steps, that is 1000 Hz, with a real-time factor of 1.0. The arena is deliberately featureless: the integration is a plumbing test for the pipeline, not a navigation benchmark, and obstacles would add a sensing problem we have intentionally not introduced yet.

Four rovers are spawned at $(-3, 0)$, $(-1, 0.4)$, $(1, 0.1)$, and $(3, 0.5)$ with small yaw offsets, all well within mutual communication range, so the initial candidate graph is complete. The rover model is a minimal differential-drive platform: a box chassis of $0.9 \times 0.65 \times 0.25$ m with mass 3.0 kg, two cylindrical wheels of radius 0.16 m and mass 0.25 kg each, placed at ± 0.38 m laterally and attached by revolute joints. The Gazebo differential-drive plugin drives the wheel joints with a wheel separation of 0.76 m and a wheel radius of 0.16 m, and publishes odometry at 20 Hz, which is the stream the bridge forwards. The model carries no IMU, no lidar, and no camera, and its odometry is ideal and noise-free. This is a conscious choice for a first integration: every quantity the planner consumes is either exactly known or explicitly scripted, so any unexpected behavior in the pipeline can be attributed to the pipeline itself rather than to sensor noise.

12.5 Limitations and Planned Remedies

In keeping with the audit stance of this project, we state what the integration does *not* yet do, and what we plan to do about it.

First, **nothing publishes velocity commands**. The planner assigns tasks, but no component translates an assignment into motion, so the rovers hold their spawn poses for the entire run and Gazebo currently contributes static geometry and an ideal odometry stream rather than genuinely dynamic behavior. The positions the pipeline reasons over are therefore constant across cycles, and only the scripted trust, load, and battery dynamics change. The planned remedy is a thin executive layer that maps each assignment to a target pose and publishes velocity commands on the differential-drive plugin's command topic, closing the loop between allocation and motion.

Second, **battery, load, capability, and link quality are scripted, not sensed**. The radio model is analytic by design for reproducibility, and the energy and load dynamics are the same deterministic scripts as in the research harness. A real deployment would estimate link quality from measured packet loss and round-trip times and read battery state from the platform; we plan to replace the analytic model with an estimator fed by the simulator's own topic statistics as a first step in that direction, keeping the analytic model as a reproducibility fallback.

Third, **the rover count is fixed at four in three places**: the bridge's topic list, the planner's subscriptions, and the world description all hard-code the fleet size, so adding a rover requires three coordinated edits and silently drops the newcomer if one is missed. The remedy is a single fleet-size parameter expanded into the bridge configuration and the world description at launch time, with the planner discovering its roster from the bridged topics.

Fourth, **there is no sim-time synchronization**: the planner ticks on a one-second wall-clock timer, so if the simulator's real-time factor drops below 1.0 under load, pipeline cycles no longer align with simulated seconds. The remedy is to drive the cycle from the simulated clock topic instead of the wall clock, which the current node structure accommodates with a one-line change to the timer source.

12.6 Running the Demonstration

The entire demonstration builds and runs with a single compose command through the launch script, which performs the disk-space check, builds the image, starts the Gazebo server and the bridge, and launches the planner node. The `STEPS` environment variable sets the number of pipeline cycles (twelve by default), after which the node reports its final status and the container stops. No ports are published and no host setup beyond Docker is required; the only artifacts are the container log and the two JSON status topics, which together record every decision the pipeline made and the state it made them from.

13 The Custom three.js Simulation Engine

The research pipeline described in the preceding chapters produces its results through batch experiments: a simulator steps through planning cycles, records outcomes, and reports statistics after the fact. Such experiments establish what the pipeline *does*, but they are poorly suited to exploring how it *behaves*: how pruned topologies respond to faults, how trust estimates track hidden reliability, or how competing assignment policies diverge when driven by identical evidence. To make these dynamics inspectable, we built an interactive browser console that drives the *real* research pipeline, comprising constrained QUBO edge pruning, trust-aware GNN scoring, greedy task assignment, and trust model updates, through a Python backend, while the browser itself owns only kinematics and rendering. The console is not a visualization bolted onto recorded data; every cycle shown on screen is produced by the same code that generated the results in this report. We regard the engine as a work in progress, and this chapter describes its current state, including explicitly marked placeholders for material that remains to be captured.

[**TO FILL: screenshot: orbit view of the fleet console with link arcs visible**]

13.1 Overview and Design Principles

The console is a single-page web application written in TypeScript with React 18, rendered through three.js via react-three-fiber, with application state held in a Zustand store and the project bundled by Vite. The backend is a deliberately dependency-free Python standard-library HTTP server, chosen so that the console runs anywhere the research code runs, without a package installation step beyond what the pipeline already needs. Three principles guided the design.

First, *the browser never plans.* All optimization, scoring, and outcome sampling happen in the Python session server; the client computes rover motion and draws the scene. This separation guarantees that what the user sees is the pipeline under test, not a reimplementation that might quietly diverge from it. Second, *one source of truth for configuration.* A shared configuration file, read by both the TypeScript client and the Python server, defines the arena geometry, vehicle kinematic limits, every tunable parameter with its scope and default, the calibrated presets, and the comparison catalogue with its published reference numbers. The two processes cannot disagree about, say, the rover's turning limit, because both read the same numbers. Third, *honest comparison by construction.* The console implements the paired-comparison methodology of the audit rather than a looser visual side-by-side, and it discloses the baselines it does not offer rather than hiding them.

13.2 Two Processes, One Command

A single development script starts both processes: the session server and the Vite development server. At runtime the two communicate over a small JSON API. The client polls `GET /api/health` at startup and periodically thereafter; the response reports which optional solvers are installed (neal, tabu, SQA, MILP), so the comparison menu only offers baselines that can actually run on the current machine. Each planning cycle is requested with `POST /api/step`, whose response carries the kept-edge topology, assignment, per-rover scores and trust values, outcome realizations, and wall-time measurements for every active baseline.

Because the console is interactive, the server is hardened against inputs a batch script would never produce. Request bodies are capped at 32 KiB; rover positions must be finite and inside the arena; every tunable parameter is range-checked against the shared configuration; and at most 8 faults may be injected per cycle. Concurrency is bounded by a semaphore admitting four simultaneous solves, with `503` returned when the server is busy, so a user dragging a slider rapidly cannot queue an unbounded solve backlog. The server hosts at most 16 sessions, each with a 600 s time-to-live and busy-window eviction, returning `429` when full. Finally, every outgoing payload passes through a sanitizer, so NaN or infinite values, which arise legitimately inside numerical

Table 18: HTTP API of the session server. All validation failures return a structured error body; no response ever contains a NaN literal.

Method and path	Error	Meaning
`GET /api/health`	200	Reports installed optional solvers and session load.
`POST /api/step`	400	Body fails validation (malformed JSON, body over 32 KiB, non-finite or out-of-arena positions, parameters out of range, more than 8 faults).
`POST /api/step`	429	All 16 session slots are occupied and none is evictable.
`POST /api/step`	503	The solve semaphore is saturated (4 concurrent solves); retry later.

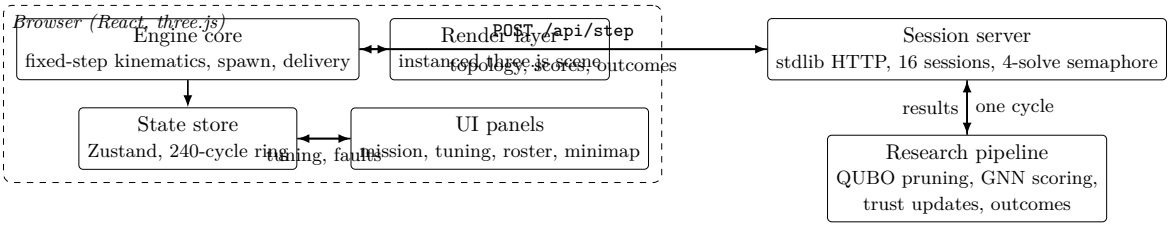


Figure 15: Console architecture. The browser owns kinematics, rendering, state, and interaction; the session server owns planning. A single shared configuration (not shown) is read by both sides.

code during degenerate configurations, can never leak into a JSON response, where they would be unparseable. Table 18 summarizes the surface.

Figure 15 shows the division of labor. The browser engine splits into four cooperating layers: a pure, testable core implementing kinematics, spawning, and delivery; a render layer built on instanced three.js geometry; a central state store; and the UI panels. Only the core talks to the network.

13.3 The Shared Configuration

The shared configuration is the contract between client and server. It fixes the physical world: a circular arena of radius 6 m, rovers with body radius 0.62 m, wheel radius 0.16 m, and track 0.76 m, limited to a maximum wheel speed of 1.25 m/s, acceleration 0.9 m/s^2, braking 1.8 m/s^2, and yaw rate 1.8 rad/s. On top of this it defines 15 tunable parameters, each annotated with a *scope* that the UI exposes to the user: session-scoped parameters restart the mission when changed (for example fleet size), planner-scoped parameters take effect on the next planning cycle (for example the QUBO budget or solve interval), and client-scoped parameters apply instantly (for example camera mode or render quality).

Seven calibrated presets ship with the console (Table 19), spanning benign field operations through a deliberately hostile communications regime. Each preset carries a measured per-cycle planning time on our reference machine, ranging from 140 ms for the lightest configuration to 1420 ms for the largest fleet, so users can see the real cost of the solver settings they choose rather than discovering it as interface lag. The same configuration also holds the comparison catalogue: for every baseline the console can run, it stores the published reference numbers from the audit, which the mission strip displays alongside live results.

Table 19: Calibrated mission presets. Planning times are measured per-cycle medians on our reference machine; where an exact value is not listed here we give the span across all presets, 140 ms to 1420 ms.

Preset	Purpose	Cycle time (ms)
field	Benign outdoor survey, default mission	140 to 300
rapid	Fast replanning, solve every cycle	140 to 400
relay	Stretched topology stressing connectivity	300 to 700
swarm	Many small rovers, dense graph	400 to 1000
fleet16	Sixteen rovers, upper end of the live fleet	900 to 1420
hostile	High packet loss, frequent faults, drift	300 to 900
tight	Small QUBO budget, aggressive pruning	140 to 500

13.4 Honest Comparison by Construction

A demo that lets the user pit our pipeline against alternatives invites an obvious failure mode: comparisons that look dramatic but are confounded. The console addresses this structurally. Assignment baselines (trust-only, random, and no-graph) do not run independent missions; they run as *paired follower simulations* that share the primary mission's kept-edge topology, so they receive identical situational evidence and differ in assignment policy only. Faults injected from the UI, whether a reliability drop, battery drain, or a packet loss spike, are applied identically to the primary and all followers, preserving the pairing. Solver baselines (bit-level SA, neal, tabu, and SQA) run on the *exact same Q matrix* the production annealer just solved, seeded with the primary's previous solution as a warm start, and the console reports for each of them the energy gap, feasibility, matched-or-better rate, and wall time.

The comparison menu also embodies the audit's honesty policy in a deliberately uncomfortable way: it excludes the two baselines that *beat* our pipeline, the exact MILP solve and the simple threshold pruner, and says so in a footnote beneath the menu rather than concealing their existence. Users exploring the console see the same negative results the paper reports.

[**TO FILL: screenshot: mission strip during a live solver comparison**]

13.5 The Engine Core

Motion and delivery are computed by a custom fixed-timestep engine written in pure TypeScript. Simulation advances at 1/120 s steps through an accumulator, with rendering interpolating between the two most recent states, so visual smoothness is decoupled from both the integration rate and the planner cadence. All simulation state lives in typed arrays rather than per-object JavaScript properties, which keeps memory layout flat, avoids per-frame allocation, and makes the hot loops friendly to the JIT.

Kinematics. Rovers are differential-drive vehicles. Rather than the common Euler approximation ($x \mathrel{+}= v\cos\theta\,\Delta t$ and so on), we integrate each step in exact constant-twist form: over a step the rover travels a circular arc determined by its linear and angular velocity, and the pose update is closed-form in the arc radius v/ω. The naive formula divides by the yaw rate ω, so we branch on $|\omega| < \varepsilon$ and evaluate the singular case through a sinc expansion, making the integrator singularity-free at zero yaw rate. Wheel-speed commands that exceed the 1.25 m/s limit are saturated by scaling *both* wheels by the same factor, which preserves curvature: the rover follows exactly the same arc, only slower, rather than veering off the intended path. An asymmetric rate limiter enforces the 0.9 m/s^2 acceleration and 1.8 m/s^2 braking limits, so rovers stop faster than they start, a small asymmetry that reads as mechanically plausible on screen.

Steering and collision avoidance. The steering controller sums several influences: seek toward the current goal, quadratic repulsion from neighboring rovers, a keep-right rule for head-on

encounters, a soft potential at the arena wall, and, for missions that want it, an optional tether pulling a rover toward its nearest network neighbor so the fleet does not silently fragment. A speed profile slows the rover into turns and as it approaches an arrival. Two hard guarantees sit beneath all of this. First, a *collision invariant*: no pair of rovers may ever come closer than two body radii. Any integration step that would violate this is rejected, and the rover rotates in place instead, so interpenetration is impossible by construction regardless of how aggressive the steering sum becomes. Second, *stuck detection*: a rover whose progress stalls executes alternating-side detours, escaping the symmetric deadlocks that pure repulsion fields are prone to.

Spawning and task placement. Missions begin from a deterministic spawn layout produced by dart throwing with a hexagonal lattice fallback, so a given fleet size and seed always yield the same starting configuration, keeping paired comparisons fair. Task sites are derived from a stable hash of the task identifier rather than from a stream of random draws, which means the same task id lands in the same place in the primary and in every follower simulation.

Delivery model. The browser reproduces the research harness's delivery semantics: message success follows a widest-path (maximum bottleneck) tree from each rover to the root rover, so the console's on-screen success statistics are the same quantity the offline simulator measures, not a decorative substitute. Task beacons carry 14 s timeouts, after which an unclaimed task expires and is recorded as a failure.

13.6 The Render Layer

Interactivity at sixteen rovers with hundreds of candidate links requires discipline about draw calls, so the render layer is *instanced everything*: one draw call per system regardless of fleet size.

All radio links, candidate, kept, breaking, and widest-path, are drawn by a single shader-drawn ribbon mesh. Each link is a 24-segment quadratic arc whose style is selected per instance: candidate links are faint, kept links solid, breaking links flicker at 14 Hz to draw the eye, and the widest-path tree is highlighted with animated packet flow moving toward the root, giving an immediate read on where the network's bottlenecks currently are. One additional instanced draw renders all ground decals: contact shadows, communication-range rings, the selection ring, fault glows, and beacon pads. Task beacons are a further instanced mesh.

The fleet itself is drawn as one instanced mesh per body part plus wheel groups. The default rover model is procedural, with dimensions taken from the physics simulator's rover description: a chassis of $0.9 \times 0.65 \times 0.25$ m on 0.16 m wheels, so the browser robot matches the simulated robot rather than an artist's impression. A custom vertex-attribute PBR technique bakes color, roughness, and metalness into vertex data, collapsing what would conventionally be a multi-material, multi-draw model into a single draw call for the entire body. Users can also import their own GLB rover models; imports are validated against 24 MB and 250,000-triangle limits, wheels are detected by node name, and the model is normalized to the standard footprint so a user-supplied robot obeys the same kinematics as the default.

The camera offers orbit, top-down, follow, and chase modes with smooth glides between them, switched from the keyboard (keys 1 to 4; space pauses the mission). The view frustum's ground footprint is projected into an RTS-style north-up minimap, so the user can see both what the main camera covers and where every rover sits. Three quality tiers adapt rendering cost to the hardware: high enables MSAA, ambient occlusion, and bloom; balanced uses SMAA with bloom; low disables postprocessing entirely. A performance monitor demotes the tier automatically when frame times sag, and promotes it again when headroom returns.

[TO FILL: performance capture: fps and draw calls at 16 rovers on the high quality tier]

13.7 State, Planner Loop, and User Interface

Application state lives in a single Zustand store holding a 240-cycle history ring, which backs the rolling charts without unbounded memory growth. The planner loop keeps exactly one request in flight at an adaptive cadence, 1.6 s by default; when the server reports congestion it backs off exponentially to 8 s rather than piling requests onto a busy backend. Independently, telemetry is copied into the store at 5 Hz so the HUD stays fluid even while the planner is mid-cycle.

The interface comprises a top bar; a mission panel with tabs for mission presets, comparisons and fault injection, tuning sliders annotated with their scope notes and graph-theory advisories (for example warning when a degree cap threatens connectivity), and view, quality, and model-import controls; a mission strip showing live statistics, including bootstrap confidence intervals over paired cycles, rolling sparklines, and plain-language significance wording in place of raw p-values; a per-rover roster whose heading-up radar canvases plot each rover's estimated trust against the hidden ground-truth reliability, exposing how fast the trust model converges after a fault; and the north-up minimap. Faults injected from the panel, reliability drop, battery drain, or packet loss spike, take effect in the primary and every follower simulation at the same cycle, preserving the paired design.

[**TO FILL: link or QR code to a demo video**]

13.8 Planned Engine Extensions

The engine is functional but incomplete, and we close with the three extensions we consider most valuable, in priority order.

WebSocket streaming for mid-cycle telemetry. The current poll-per-cycle design hides everything that happens inside a solve. A WebSocket channel would stream intermediate annealer energies, trust updates, and assignment proposals as they occur, turning the console into an instrument for observing solver dynamics rather than only their endpoints.

Session persistence and mission replay. Sessions currently evaporate with their 600 s TTL. Recording the full request and response stream would support resuming interrupted sessions and replaying missions deterministically, which would also simplify regression testing of the client against recorded server behavior.

Motion command publishing. At present the physics simulator and the browser engine run separate world models with matched parameters. Publishing motion commands from one engine to the other, so both drive the same rovers, would remove the residual risk of kinematic divergence and enable mixed-reality demonstrations in which simulated and browser-rendered fleets interleave.

14 Conclusion and Future Work

14.1 What was built

This project set out to answer a practical question: can a constrained combinatorial encoding of graph pruning, combined with trust-aware learning on the surviving topology, improve task allocation in a dynamic rover network? To answer it honestly we built the complete pipeline rather than a toy demonstration. The system consists of a constrained QUBO formulation of edge selection with degree, budget, flow connectivity, and flow capacity constraints; a feasibility-projected production annealer that moves only in the space of valid edge sets; a trust-aware graph neural network that scores rovers on the pruned topology; a capacity-aware greedy assignment stage; and an online trust model that folds each mission's outcomes back into the next cycle's

features. Around this core we built the apparatus needed to audit it: a causal simulation harness in which mission success genuinely depends on the kept topology, an exact MILP baseline for ground truth, a hostile test suite of 491 checks organised into fourteen sections, a physics integration based on Gazebo Harmonic and ROS 2 Jazzy, and an interactive browser console driven by a custom three.js simulation engine that runs the same allocation logic against a live visualisation.

14.2 What was proven

The QUBO encoding itself is verified three independent ways: an independent re-derivation of the energy function agrees with the constructed Q matrix term by term; brute-force enumeration of all edge subsets on small instances reproduces the encoding's optima; and an exact linearized solve of the matrix with CBC confirms the same energies. The production annealer is 100% exact on small instances against enumeration, 95% exact against MILP on twenty larger instances with a mean true optimality gap of 7×10^{-5}, and 87% exact across all sensitivity perturbations of temperatures, cooling, restarts, penalty scales, and move probabilities. At the system level, trust-aware assignment beats random assignment by +13.8 percentage points of mission success and a trust-only heuristic by +8.13 points, with a paired bootstrap 95% confidence interval of $[+4.2, +11.7]$ and a permutation p-value of 0.016. Trust tracking itself is worth +13.75 points of staleness margin: the benefit persists when topology decisions are amortised over several cycles, which matters when the solve must be offloaded to a cloud tier.

14.3 What was disproven or humbled

An honest audit must report what did not work, and this project treats those results as first-class findings. First, classical MILP solved by CBC is exact and roughly one hundred times faster than the annealer on the same instances. The QUBO formulation is therefore a hardware-target encoding, not the recommended classical solver, and the paper says so without qualification. Second, four serious annealing samplers, a bit-level simulated annealer, a production simulated annealing library, a tabu search, and simulated quantum annealing, reached the optimum on the penalty-encoded landscape only 48% to 64% of the time, and random bit-string restarts returned infeasible states in 19 of 25 instances. Third, at the ten-rover scale a simple threshold pruner beats the QUBO pruner on raw mission success, 38.2% to 33.8% with $p = 0.028$, because the threshold keeps redundant links that rescue delivery through bottleneck topologies. Fourth, message passing adds only +1.7 insignificant points over a no-graph readout, so the graph mechanism's value is not yet established at this scale. Fifth, a cloud-trained readout is 3.9 points worse than the fixed hand-designed weights ($p = 0.49$), a clean null result against the trainability hypothesis.

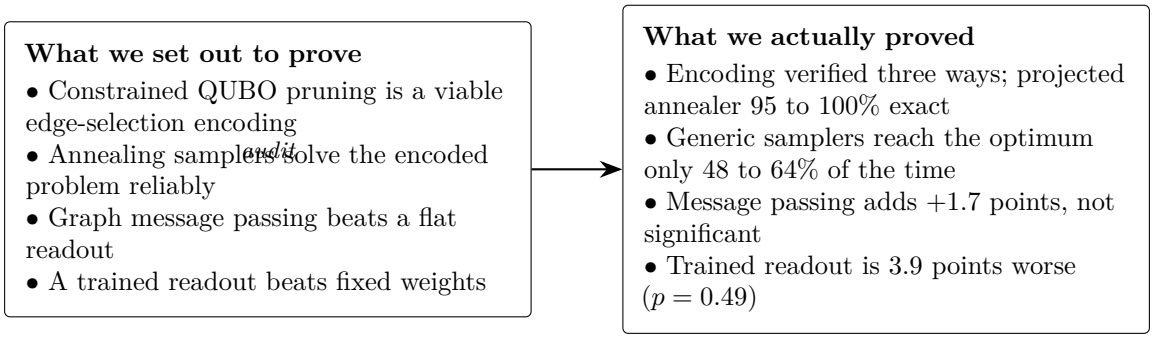


Figure 16: The distance between the proposal and the audited result. Every item on the left survived contact with measurement in a modified form; the right column is what the evidence supports.

14.4 Lessons

Four lessons generalise beyond this project. First, verify encodings against independent ground truth before trusting them: our encoding survived because it was cross-checked against re-derivation, enumeration, and an exact solver, and that triple verification is what lets the downstream claims stand. Second, penalty-encoded constrained QUBOs are a hard landscape for generic samplers; the penalty barriers that enforce feasibility are exactly what trap samplers that move in raw bit space. Third, feasibility-aware move sets matter more than solver sophistication: the projected annealer's advantage came from proposing only valid moves and repairing candidates, not from a cleverer temperature schedule. Fourth, negative results deserve the same prominence as positive ones, because the negative results here, the MILP dominance, the threshold pruner's win, and the readout-training null, are the findings most likely to save a future team from repeating the same detours.

14.5 Future work

Several directions follow directly from what the audit exposed. The Gazebo integration should publish velocity commands so that physics drives the planning inputs rather than replaying scripted poses, closing the loop between the simulator and the allocator. The browser console needs WebSocket streaming in place of request-response stepping, plus session persistence so long missions survive reconnects. The pruning formulation should be extended to hierarchical or clustered variants, since connectivity is currently enforced only up to twenty rovers due to the dense memory footprint of the matrix. The Q matrix should be run on real annealing hardware from D-Wave to test whether the hardware changes the sampler-reliability picture that the classical samplers painted. The readout-training null result deserves a revisit with larger and more diverse training data, since the current failure may be a data problem rather than a model problem. Finally, the deterministic link-quality model should be replaced with richer radio models that include fading and interference, which would stress the trust model in ways the current environment cannot.

14.6 Closing remarks

Building this system taught the team that the distance between a plausible pipeline and a trustworthy one is measured in verification infrastructure, not in model complexity. The parts of the project we are most confident in are not the cleverest components; they are the components pinned down by independent ground truth, and the parts we trusted too early were the ones that measurement later humbled. We set out to show that a quantum-inspired encoding could earn its place in a rover network, and we ended up with something more useful: a complete, audited picture of where the encoding helps, where it does not, and exactly how much each design decision is worth. That picture, including its negative regions, is the contribution.

14.7 Pending items in this report

The following placeholders remain in this report and must be completed before submission:

- title page author names and date;
- the abstract text;
- Gazebo world and status-message screenshots;
- fleet console screenshots (orbit view, mission strip), performance capture, and demo video link;
- the reference list for the final section (to be supplied separately).

References

[**TO FILL: reference list for the final section; the authors will supply the references separately. Suggested entries to confirm: Sabater and Sierra (2005) on trust and reputation, the CBC user guide, the D-Wave sampler documentation, the OpenJij paper, Gazebo Harmonic and ROS 2 Jazzy documentation, and the three.js and React documentation.**]

A Pending Fields and Placeholders

This appendix collects every placeholder left in the report so that nothing is forgotten before submission. Each entry also appears inline at the point where the material belongs, marked as [**TO FILL: ...**].

Identity and front matter

- Title page: author names [**TO FILL: author names, one per line**].
- Title page: submission date [**TO FILL: submission date**].
- Abstract: [**TO FILL: abstract text; 150 to 250 words**].

Figures and media to add

- Gazebo chapter: screenshot of the world with the four rovers.
- Gazebo chapter: screenshot or log excerpt of a live status message.
- Engine chapter: screenshot of the orbit view with link arcs visible.
- Engine chapter: screenshot of the mission strip during a live solver comparison.
- Engine chapter: performance capture (fps, draw calls, triangles) at 16 rovers on the high quality tier.
- Engine chapter: link or QR code to a demo video.

Data cells to confirm

- Results chapter ablation table: confidence interval and p-value cells for the no-graph and random arms [**TO FILL: CI and p for these two arms, if they should be printed**].
- Results chapter pruner table: exact success rates for the full-graph and top-K arms [**TO FILL: exact rates, if they should be printed**].
- Engine chapter preset table: per-preset measured cycle times are currently ranges inside the measured 140 to 1420 ms span; replace with exact per-preset measurements if they should be printed.

References

- The reference list for the final section is to be supplied separately by the authors. See the References placeholder section.